



Abschlussprüfung – Sommer 2024

Fachinformatiker für Anwendungsentwicklung

Dokumentation zur betrieblichen Projektarbeit mit dem Thema

Coffee Helper

Kaffee-Zubereitungs-Helfer-Tool in Python mit Beeware

Neckargemünd, den 04.09.2024

Prüfungsteilnehmer:

Lukas Brackmann

Ausbildungsbetrieb:

SRH Berufsbildungswerk Neckargemünd GmbH

Im Spitzerfeld 25

69151 Neckargemünd



Inhalt

1	Einleitung.....	1
1.1	Projektbeschreibung	1
1.2	Projektziel	1
1.3	Projektumfeld.....	2
1.4	Projektbegründung.....	2
1.5	Projektschnittstellen	2
2	Projektplanung.....	4
2.1	Projektphasen	4
2.2	Ressourcenplanung	5
2.2.1	Zeitraahmen	5
2.2.2	Hardwarekosten.....	5
2.2.3	Arbeitskosten.....	5
2.2.4	Nebenkosten	5
2.2.5	Raumkosten.....	5
2.2.6	Gesamtentwicklungskosten	5
2.3	Entwicklungsprozess.....	6
3	Analysephase.....	7
3.1	Ist-Analyse	7
3.2	Wirtschaftlichkeitsanalyse	7
3.2.1	„Make-or-Buy“-Entscheidung.....	7
3.2.2	Projektkosten	7
3.2.3	Amortisationsdauer	7
3.3	Nicht-monetäre Vorteile	8
3.4	Anwendungsfälle.....	8
4	Entwurfsphase	10
4.1	Zielplattform	10
4.2	Architekturdesign	10
4.3	Entwurf der Benutzeroberfläche	12
4.4	Pflichtenheft	12
5	Implementierungsphase.....	13
5.1	Entwicklungsumgebung	13



5.2	Zielformat	13
5.3	Entwicklung	14
5.3.1	Projekterstellung	14
5.3.2	Erstellung der GUI	14
5.3.3	Geschäftslogik	16
5.3.4	Dateiformat für Zubereitungsmethode	18
5.3.5	Packaging	18
5.3.6	Testing	19
6	Abnahme- und Einführungsphase	20
6.1	Abnahme durch den Fachbereich	20
6.2	Deployment und Einführung	20
7	Dokumentation	21
8	Fazit	22
8.1	Soll-/Ist-Vergleich	22
8.2	Lessons Learned	22
8.3	Ausblick	23
9	Quellen	24
10	Glossar	25
11	Kundendokumentation	I
11.1	Installation	I
11.2	Bedienung	II
11.3	Einstellungen	VII
12	Auszug aus dem Testprotokoll	VIII
12.1	Explorative Testing	VIII
12.1.1	Standardnutzungsfall	VIII
12.1.2	Ungewöhnliche Eingabewerte	VIII
13	Quellcodedokumentation	IX
13.1	Dateistruktur	IX
13.2	Code	X
13.2.1	app.py	X
13.2.2	main_container.py	XI
13.2.3	recipe.py	XV



13.2.4	screens.py	XVI
13.2.5	settings_window.py	XXII
13.2.6	timer.py.....	XXIII
13.2.7	calcs.py	XXV
13.2.8	constants.py	XXVI
13.2.9	json_reader.py.....	XXVI
13.2.10	settings.py.....	XXVII
13.2.11	test_calcs.py	XXVIII
13.2.12	aeropress.json.....	XXX
13.2.13	frenchpress.json.....	XXXI



1 Einleitung

Die folgende Projektdokumentation beschreibt den Ablauf des IHK-Abschlussprojekts, welches der Autor in Rahmen seiner Ausbildung zum Fachinformatiker für Anwendungsentwicklung beim SRH Berufsbildungswerk Neckargemünd durchgeführt hat. Das Projekt geschieht im Auftrag der Simply Coffee GmbH, einem kleinen Coffee-Shop in Heidelberg.

1.1 Projektbeschreibung

Die Simply Coffee GmbH verkauft Specialty Coffee, also Kaffee von besonderer Qualität und oft gesicherter Herkunft, als ganze Bohne oder in gemahlener Form. Die Kunden können diesen entweder im Café vor Ort oder bei sich zu Hause konsumieren. Das im Projekt entwickelte Programm soll den Kunden dabei helfen, selbst Kaffee zuzubereiten.

Die Umsetzung soll in Python mithilfe des „Beeware“-Projekts umgesetzt werden. Dieses ermöglicht einfaches Deployment auf verschiedenen Betriebssystemen. Im Rahmen dieses Projektes wird zunächst nur für Windows entwickelt. Das Endprodukt soll ein MSI-Installer für Windows sein. Es besteht in Folgeprojekten die Möglichkeit, das Programm auf andere Betriebssysteme zu übertragen.

Die Verbreitung soll über einen Download-Link auf der Webseite der Simply Coffee GmbH erfolgen. Dieser wird durch QR-Codes im Geschäft und auf Produktverpackungen beworben.

Folgende Features sollen unterstützt werden:

1. Benutzerfreundliche GUI
2. Zubereitung mit French Press und Aeropress
3. Berechnung von Kaffeemenge basierend auf Wassermenge und umgekehrt
4. Step-by-Step-Anweisungen zur Zubereitung
5. Timer für Wartezeiten, z.B. Ziehzeiten
6. Konfiguration: Kaffeemühle oder vorgemahlener Kaffee

1.2 Projektziel

Ziel des Projektes ist es, dem Kunden eine einfache Möglichkeit zu geben, die Produkte der Simply Coffee GmbH optimal zuzubereiten. Viele Kunden haben noch nie von Hand Kaffee zubereitet und sind daher verunsichert. Das Programm ermöglicht es, die Kunden Schritt für Schritt an den Prozess heranzuführen. Dadurch ist die Wahrscheinlichkeit, dass die Kunden ein positives Erlebnis haben, größer. Die Marketingabteilung erwartet, dass dies zu mehr Wiederholungskäufen führt. So sollen die Projektkosten refinanziert werden.



1.3 Projektumfeld

Der Auftraggeber ist die Simply Coffee GmbH. Die Simply Coffee GmbH ist ein kleiner Coffee-Shop in Heidelberg mit 10 Mitarbeitern. Der Kontakt ist durch eine Anzeige vor Ort, die der Autor als Kunde wahrgenommen hat, entstanden. Der Ansprechpartner ist der Store Manager Herr Bohne.

Der Coffee-Shop verkauft Specialty Coffee, also Kaffee, der von besonders hoher Qualität ist und oft gesicherte Herkunft hat. Der Betrieb zeichnet sich durch Offenheit gegenüber modernen Technologien aus. Er bietet bereits einen Online-Shop an und möchte nun sein Angebot um eine Companion-App erweitern, die es den Kunden leichter macht, ihren Kaffee zuzubereiten. Die App soll (im Anschluss an das Projekt) speziell auf das Branding und die Kaffees von Simply Coffee zugeschnitten werden. Dadurch können auch neue Kunden auf den Online-Shop geworben werden. Da ein großer Teil der Kundschaft internationale Studenten sind, wird die App auf Englisch entwickelt.

Das Projekt wird im Rahmen der betrieblichen Projektarbeit in der SRH Neckargemünd erstellt.

1.4 Projektbegründung

Wie bereits erläutert verkauft die Simply Coffee GmbH Specialty Coffee. Dieser ist deutlich teurer als Supermarktkaffee. Damit sich der preisliche Unterschied auch im Geschmack widerspiegelt, muss er richtig zubereitet werden. Viele Kunden haben beklagt, dass sie nicht wissen, wie sie ihren Kaffee so zubereiten können, dass er so gut wie im Geschäft schmeckt. Die Marketingabteilung hat vorgeschlagen, eine App zu entwickeln, die die Endkunden durch maßgeschneiderte Zubereitungsschritte führt, um immer ein optimales Ergebnis zu erzielen. So soll eine größere Kundenzufriedenheit erzielt werden, die die Kunden dazu bewegt, erneut bei Simply Coffee einzukaufen.

1.5 Projektschnittstellen

Das Programm wird auf Basis des Beeware-Projekts erstellt. Dieses wird verwendet, um das Projekt anzulegen, die Datenstruktur zu erstellen und das Programm zu einer installierbaren MSI-Datei zu verpacken. Zudem wird das zugehörige Toga-Frontend-Framework verwendet.



Python-Logo



*Brutus, das Maskottchen
von Beeware*



Es werden statische Daten, die die verschiedenen Zubereitungsmethoden darstellen, verwendet. Diese werden aus JSON-Dateien ausgelesen. Hierzu wird das Standard-Python-Modul „JsonReader“ verwendet.

Die Programmeinstellungen werden in einer INI-Datei gespeichert. Diese wird vom Standard-Python-Modul ConfigParser ausgelesen und geschrieben.

A screenshot of a text editor window titled "config.ini - Editor". The window has a menu bar with "Datei", "Bearbeiten", "Format", "Ansicht", and "Hilfe". The main text area contains the following text:

```
[Hardware]
coffeegrinder = False
```

The status bar at the bottom shows "Zeile 4, Spalte", "100%", "Windows (CRLF)", and "UTF-8".

Inhalt der config.ini



2 Projektplanung

2.1 Projektphasen

Kundengespräche: Es fand ein intensiver Austausch mit dem Kunden statt, um die Anforderungen zu verstehen und gemeinsame Ziele festzusetzen.

Planung: Der Ablauf des Projektes, die Ressourcen und der Entwicklungsprozess wurden geplant.

Analyse: Es wurde eine genaue Aufnahme des Ist-Zustands gemacht sowie eine umfangreiche Analyse der Wirtschaftlichkeit des Projektes durchgeführt.

Implementierung: Hier wurde das tatsächliche Projekt umgesetzt. Der Benutzeroberfläche wurde entworfen sowie der Programmcode umgesetzt und getestet.

Erstellen der Dokumentation: Der gesamte Entwicklungsprozess wurde detailliert dokumentiert. Zudem wurde ein Benutzerhandbuch für den Endkunden erstellt.

Kundenübergabe: Das fertige Programm wurde an den Kunden übergeben und in die vorhandenen Systeme eingeführt. Dem Kunden wurde der Umgang mit der Software gezeigt.

Soll- / Ist-Vergleich: In dieser Phase wurde geprüft, ob die Ziele des Projekts erreicht wurden.

Projektphase	Geplante Zeit
Kundengespräche	2 h
Planung	8 h
Analyse	6 h
Implementierung	30 h
Erstellen des Projekts in Beeware	1 h
Erstellen der GUI	12 h
Implementierung der Geschäftslogik	11 h
Testing / Debugging	5 h
Building / Packaging	1 h
Erstellen der Dokumentation	28 h
Projektdokumentation	25 h
Kundendokumentation	2 h
Quellcodedokumentation	1 h
Kundenübergabe	2 h
Soll- / Ist-Vergleich	4 h
Summe	80 h



2.2 Ressourcenplanung

Im Folgenden wird analysiert, welche Ressourcen für die Entwicklung des Projekts benötigt werden. Die monetären Werte wurden von der Buchhaltung zur Verfügung gestellt.

2.2.1 Zeitrahmen

Das Projekt wird am 16.07.2024 begonnen und wird am 23.08.2024 abgeschlossen. Die Arbeitszeit beträgt 80 Stunden.

2.2.2 Hardwarekosten

Zur Entwicklung wird ein Arbeitsrechner benötigt, der Anteilig für die Projektzeit berechnet wird. Die Anschaffungskosten betrugen 1248 €. Es wird mit einer Nutzungsdauer von 2 Jahren gerechnet. Da das Projekt auf 80 h ausgelegt ist, wird basierend auf 40 h-Wochen von einer Nutzungsdauer von 2 Wochen gerechnet.

$$\begin{aligned} \text{Hardwarekosten} &= \frac{\text{Anschaffungskosten}}{52} * \text{Nutzungswochen} \\ \frac{1248,00 \text{ €}}{52} * 2 &= 48,00 \text{ €} \end{aligned}$$

Der Computer wird anteilig mit 48 € berechnet.

2.2.3 Arbeitskosten

Der Stundenlohn der Entwickler wird mit 75,50 € / Stunde berechnet.

$$\begin{aligned} \text{Arbeitskosten} &= \text{Stundenlohn} * \text{Arbeitsstunden} \\ 75,50 \text{ €} * 80 &= 6040,00 \text{ €} \end{aligned}$$

Die Arbeitskosten belaufen sich auf 6040,00 €.

2.2.4 Nebenkosten

Die Nebenkosten betragen 100,00 € pro Woche. Bei einer Projektdauer von 2 Wochen werden diese mit 200,00 € berechnet.

2.2.5 Raumkosten

Die Raumkosten betragen 175,00 € pro Woche. Bei einer Projektdauer von 2 Wochen werden diese mit 350,00 € berechnet.

2.2.6 Gesamtentwicklungskosten

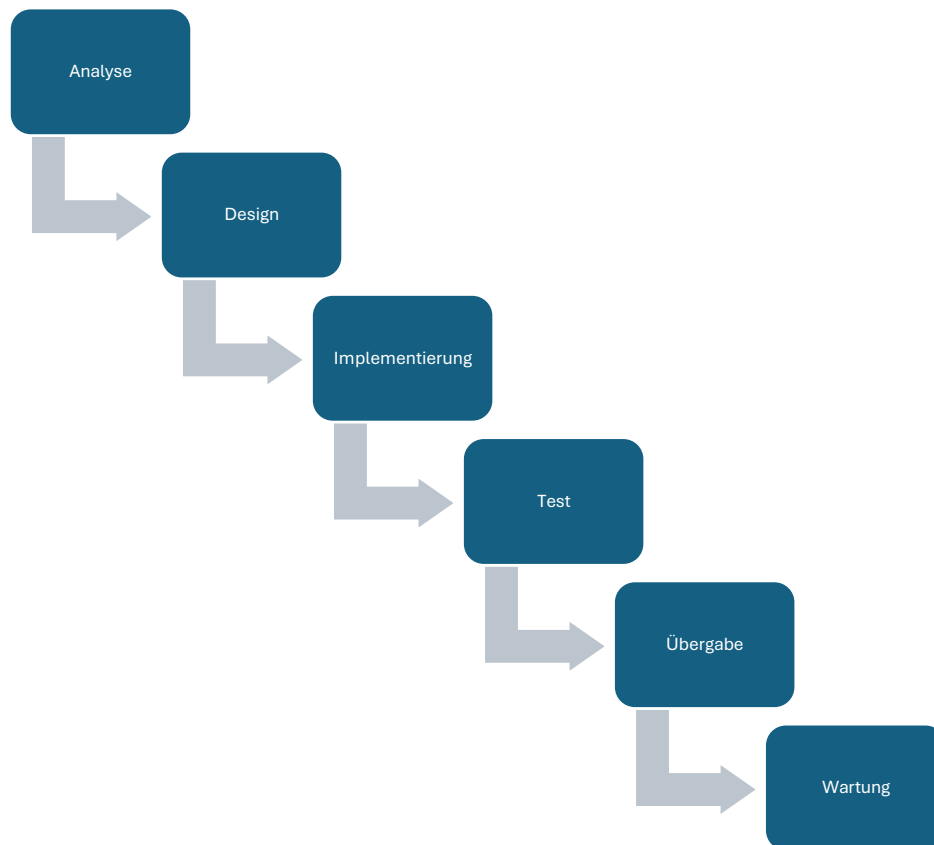
$$\begin{aligned} \text{Entwicklungskosten} &= \text{Hardwarekosten} + \text{Arbeitskosten} + \text{Nebenkosten} + \text{Raumkosten} \\ 48,00 \text{ €} + 6040,00 \text{ €} + 200,00 \text{ €} + 350,00 \text{ €} &= 6638,00 \text{ €} \end{aligned}$$

Die gesamten Entwicklungskosten belaufen sich auf 6638,00 €.



2.3 Entwicklungsprozess

Da es sich um eine einzelne Applikation mit einem sehr genau vorgegebenen Funktionsumfang handelt, haben sich der Entwickler und der Kunde auf eine traditionelle Entwicklungsmethode gegenüber z.B. agiler Softwareentwicklung geeinigt. Zur Planung dieser wurde ein Wasserfallmodell erstellt. Dies ist im Folgenden abgebildet.



Jeder Entwicklungsschritt ist in sich geschlossen. Der nächste Schritt wird jeweils erst nach Abschluss des vorherigen Tests begonnen. Da bereits während der Implementierungsphase getestet wurde, folgte das Projekt dem strengen Wasserfallmodell nicht vollständig.

In der Entwicklung wurde großer Wert auf Sorgfalt und Fehlerfreiheit gelegt. Die Funktionalität des Programms wurde regelmäßig mit Unittests und manuellem Testen überprüft.



3 Analysephase

3.1 Ist-Analyse

Die Simply Coffee GmbH betreibt eine Webseite und einen Online-Shop. Es gibt derzeit keine Software, die den Kunden direkt bei der Zubereitung unterstützt. Kunden können sich zwar vom Personal beraten lassen, wissen aber oft nicht, wie sie ihren Kaffee zubereiten können. Vor allem Neukunden bzw. Kunden, die noch wenig Kontakt mit Specialty Coffee hatten und keine teuren Kaffeemaschinen besitzen wissen oft nicht, wie sie ihn zu Hause zubereiten können.

Das Personal hat bis jetzt oft zur Anschaffung und Benutzung von French Press oder Aeropress geraten, da beide kostengünstig und einfach in der Bedienung sind. Doch oft wissen die Kunden nicht, wie sie diese bedienen und gute Ergebnisse mit den Produkten der Simply Coffee GmbH erzielen können.

3.2 Wirtschaftlichkeitsanalyse

3.2.1 „Make-or-Buy“-Entscheidung

Auf dem Markt gibt es viele kostenlose und kostenpflichtige Anwendungen, die versprechen, dem Anwender dabei zu helfen, Kaffee zuzubereiten. Das Ziel des Projektes ist es jedoch, die Kunden der Simply Coffee GmbH zu binden, sowie das Branding des Unternehmens in die Anwendung einbinden zu können. Zudem soll sichergestellt werden, dass die Zubereitungsmethoden perfekt zu dem angebotenen Kaffeesortiment passen. Daher wurde entschieden, das Programm selbst zu entwickeln.

3.2.2 Projektkosten

Wie bereits in der [Ressourcenplanung](#) ermittelt, betragen die Entwicklungskosten 6638,00 €. Hinzu kommen die Personalkosten fürs Fachgespräch von 120 €, Kosten für die Übergabe von 90 €, sowie Schulungskosten von 115 €. Diese Kosten wurden von der Finanzabteilung vorgegeben.

$$\begin{aligned} \text{Gesamtkosten} &= \text{Entwicklungskosten} + \text{Fachgespräch} + \text{Übergabe} + \text{Schulung} \\ 6638,00 \text{ €} + 120,00 \text{ €} + 90,00 \text{ €} + 115,00 \text{ €} &= 6963,00 \text{ €} \end{aligned}$$

Die Gesamtkosten belaufen sich somit auf 6963,00 €.

3.2.3 Amortisationsdauer

Die Marketingabteilung erwartet, dass die Einführung der App zu einer Erhöhung der Verkaufszahlen führt. Die Kunden werden den Kaffee besser zubereiten können und dadurch ein positiveres Erlebnis haben. Das wird sie zu erneuten Käufen bewegen.



Die Abteilung schätzt, eine Umsatzsteigerung von 1 % erzielen zu können. Die Buchführung hat einen durchschnittlichen monatlichen Umsatz von 248.600 € bestimmt. Hieraus errechnet sich folgender wirtschaftlicher Vorteil:

$$\text{Umsatzsteigerung absolut} = \text{monatlicher Umsatz} * \text{Umsatzsteigerung \%}$$

$$248.600,00 \text{ €} * 1 \% = 2486,00 \text{ €}$$

Auf dieser Basis konnte die Amortisationsdauer berechnet werden. Der Rückfluss ist in diesem Falle die Umsatzsteigerung durch die erhöhten Verkaufszahlen.

$$\text{Amortisationsdauer} = \frac{\text{Projektkosten}}{\text{monatlicher Rückfluss}}$$

$$\frac{6963,00 \text{ €}}{2486,00 \text{ €}} = 2,8 \text{ Monate}$$

Die Projektkosten haben sich nach 2,8 Monaten amortisiert.

3.3 Nicht-monetäre Vorteile

Durch die Einführung der Software wird erwartet, dass die Zufriedenheit der Endkunden zunimmt, da die Ergebnisse der Zubereitung der Produkte besser sind. Zudem erwartet die Marketingabteilung, dass sich die Einführung und Bewerbung der Anwendung positiv auf das Markenimage auswirken werden, da die Verbindung von Produkten mit Software zum modernen Erscheinungsbild der Firma beiträgt.

3.4 Anwendungsfälle

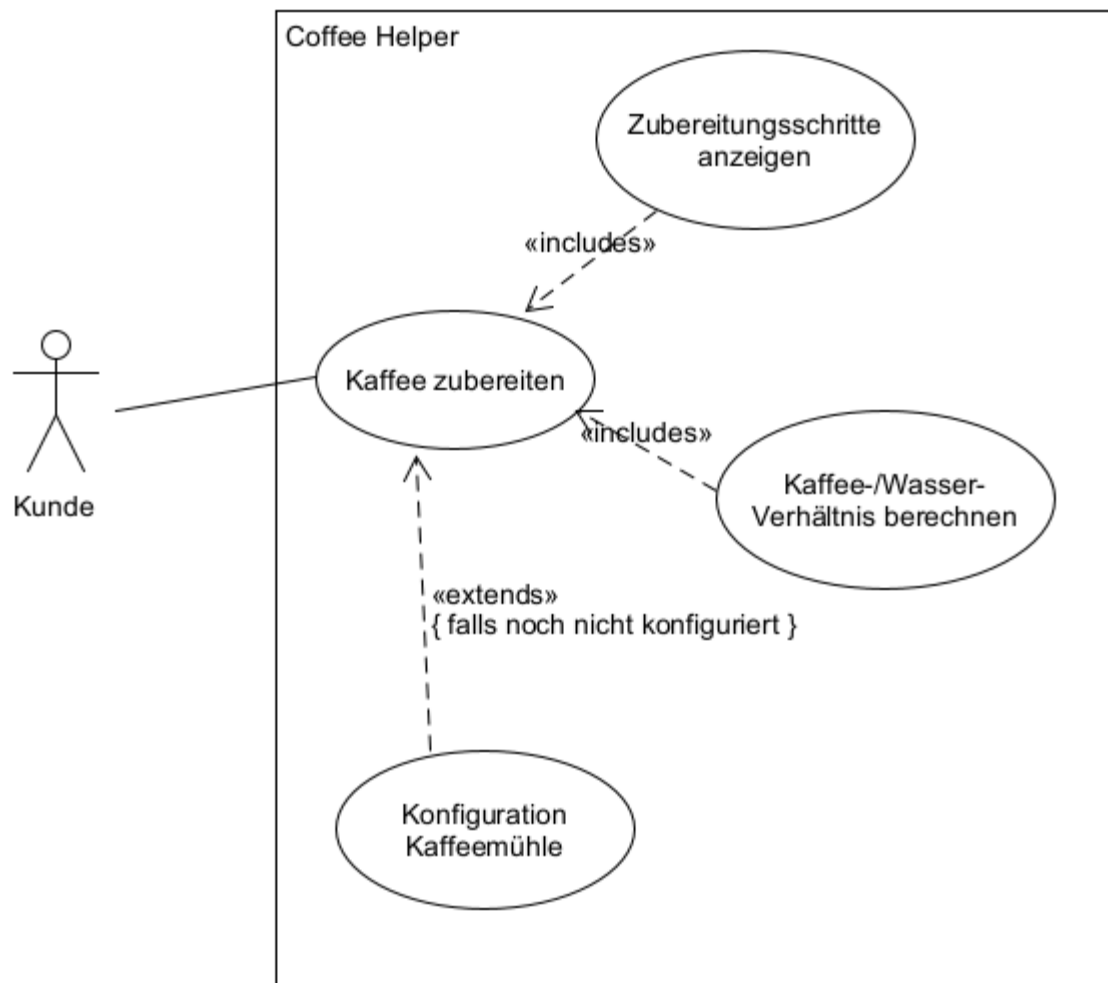
Ein typischer Anwendungsfall könnte beispielhaft folgendermaßen ablaufen:

Ein Kunde geht in den Coffee-Shop der Simply Coffee GmbH. Er kauft eine Packung lose Kaffeebohnen und wird vom Verkäufer auf den QR-Code auf der Verpackung aufmerksam gemacht. Er scannt ihn und installiert die Anwendung auf einem kompatiblen Gerät.

Der Kunde nutzt das Programm bei der nächsten Zubereitung seines Kaffees. Er folgt den Anweisungen und bereitet den Kaffee korrekt zu. Da die Anweisungen perfekt auf das Produkt zugeschnitten sind und leicht zu folgen sind, erhält er ein sehr gutes Ergebnis. Da ihm der Kaffee schmeckt und er ein sehr positives Ergebnis hatte, geht er das nächste Mal wieder zu Simply Coffee.

Es ist auch möglich, dass der Kunde im Gespräch mit dem Verkäufer anmerkt, dass er keine French Press oder Aeropress besitzt. In diesem Falle kann er möglicherweise überzeugt werden, vor Ort eine zu kaufen. Aufgrund des günstigen Kaufpreises ist ein Spontankauf hier denkbar.

Im Folgenden ist der typische Anwendungsfall als Use-Case-Diagramm dargestellt.



Anwendungsfall-Diagramm (UML)



4 Entwurfsphase

4.1 Zielplattform

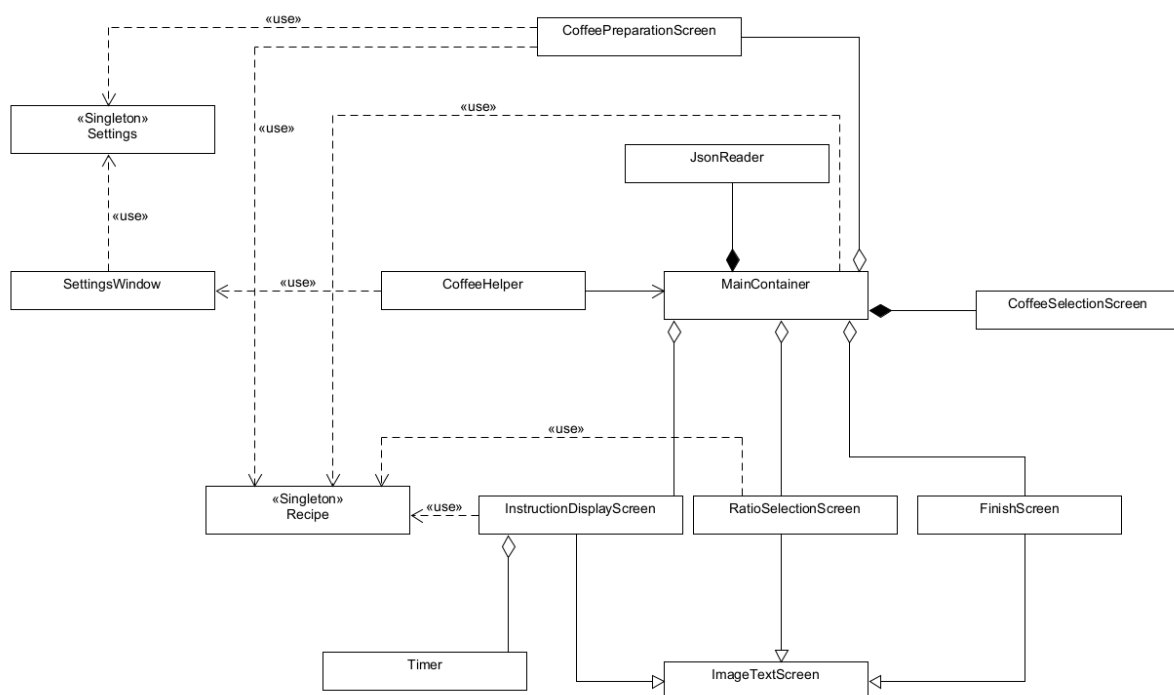
Das Programm soll zunächst auf Windows zur Verfügung stehen. Hierfür wird das Beeware-Software-Bundle und das zugehörige Front-End-Framework Toga verwendet. Dieses ermöglicht die Kompilierung in Installationsdateien für verschiedene Plattformen. Langfristig soll das Programm auch auf anderen Plattformen zur Verfügung stehen, das ist jedoch nicht Teil des Umfangs dieses Projektes.

Für Windows wird das Programm als MSI-Installationsdatei zur Verfügung gestellt. Diese kann einfach auf einer beliebigen Windows-Maschine ausgeführt werden, um das Programm zu installieren. Darüber hinaus muss eine aktuelle Version des .NET Frameworks installiert sein. Sollte dies nicht der Fall sein, führt die Installationsdatei den Nutzer automatisch durch die Installation des Frameworks. Hierfür werden jedoch Administrator-Rechte benötigt.

4.2 Architekturdesign

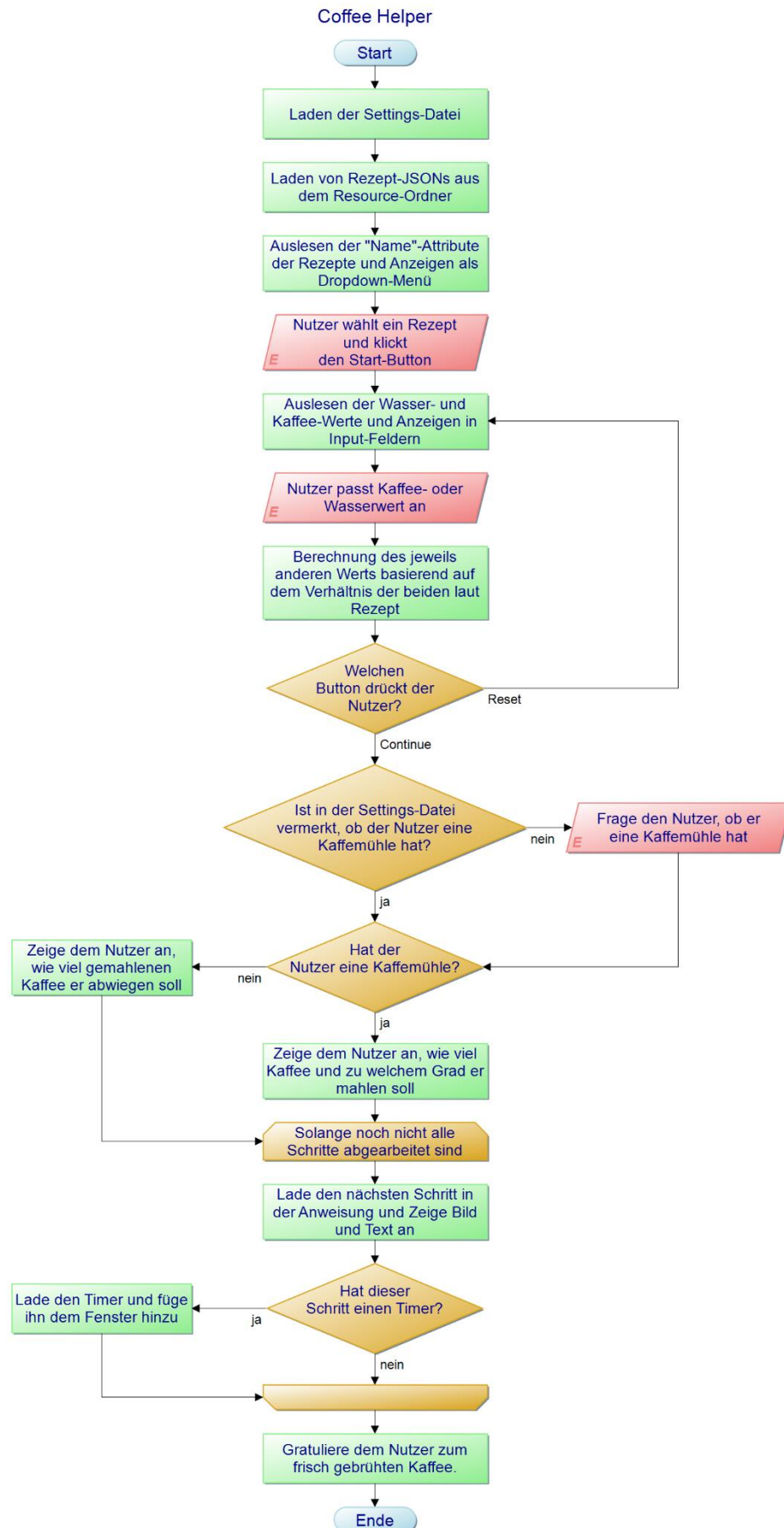
Das Projekt wurde gemischt, jedoch vorwiegend objektorientiert erstellt. Hierzu wurde die modulare Natur von Python zum vollen Potenzial genutzt, indem ähnliche Klassen zusammen in ein Modul ausgelagert wurden und bestimmte, unabhängige Elemente wie z.B. globale Konstanten ebenfalls in Module platziert wurden.

Die grobe Architektur ist hier anhand eines vereinfachten UML-Klassendiagramms dargestellt.





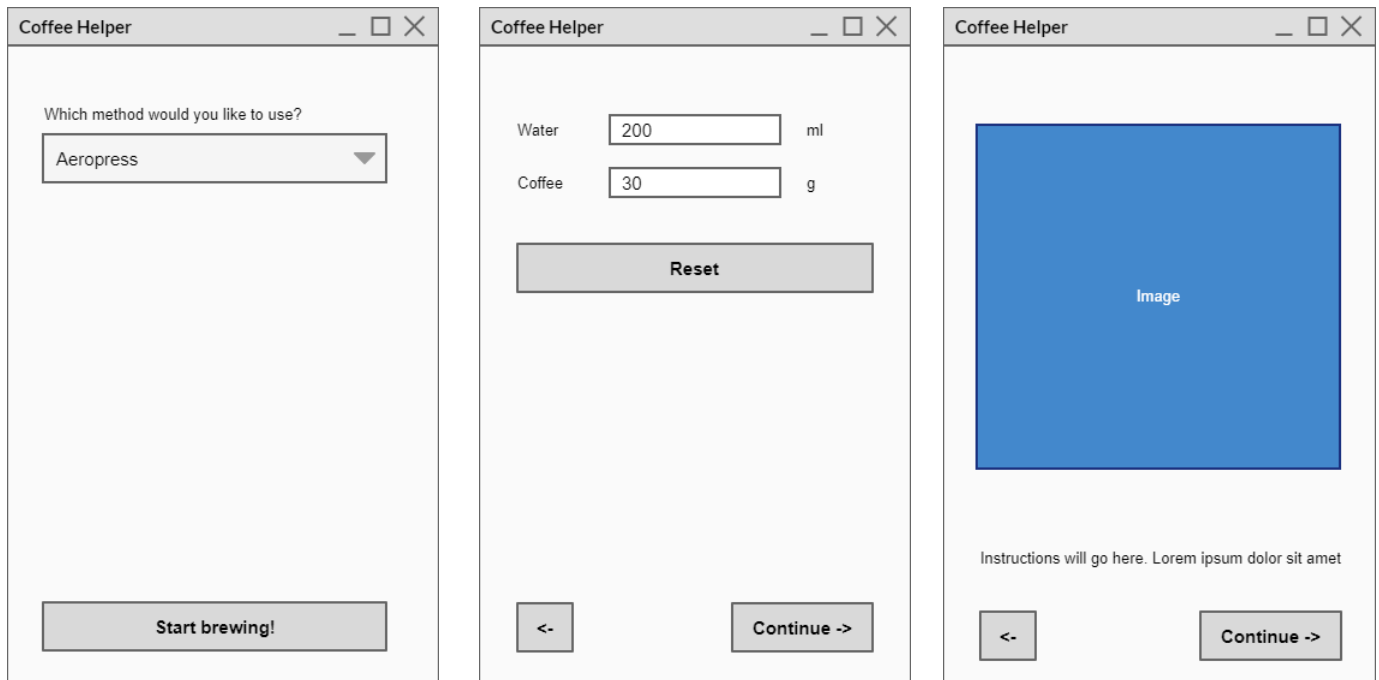
Im Folgenden ist der grobe Ablauf des Programms als Programmablaufplan (PAP) dargestellt:





4.3 Entwurf der Benutzeroberfläche

Ein wichtiges Ziel des Projekts war, eine simple, intuitive und übersichtliche Benutzeroberfläche zu erstellen. Um dies sicherzustellen, wurden zunächst Mockups erstellt, die den grundlegenden Aufbau der GUI darstellen. Hierzu wurde die kostenlose Software „Pencil“ verwendet.



Auswahlbildschirm

Wasser- / Kaffee-Eingabe

Anweisungsschritte

4.4 Pflichtenheft

Am Ende der Entwurfsphase wurde basierend auf den erarbeiteten Anforderungen ein Pflichtenheft erstellt. Es stellt die Grundlage des durchzuführenden Projektes dar. Es folgt ein Auszug aus dem Pflichtenheft.

Erstellen des Projektes und Version Control	Projektdaten werden mit Beeware erstellt, Version Control mit Git
Entwurf der Benutzeroberfläche	Mockups mit Pencil, Umsetzung in Toga
Erstellen des JSON-Formats für Rezepte	Festlegung der Dateistruktur, Umsetzung von Beispieldaten
Erstellung des Ladeprozesses	Erstellen einer JSON-Reader-Klasse, die die Datei ausliest und verifiziert
Umsetzung der allgemeinen Geschäftslogik	Die Geschäftslogik soll in Python erstellt werden. Hierbei soll Gebrauch von Klassen und Modulen gemacht werden.
Packaging und Übergabe	Die Anwendung soll als Windows-Installationsdatei zur Verfügung gestellt werden. Der Installer wird auf der Webseite der Simply Coffee GmbH zum Download angeboten werden.



5 Implementierungsphase

5.1 Entwicklungsumgebung

Die Entwicklung wurde an einem PC des SRH Berufsbildungswerk Neckargemünd durchgeführt. Er hat einen Intel Core i5-11600K-Prozessor sowie 16 GB RAM.

Als Entwicklungsumgebung wurde der Code-Editor Visual Studio Code verwendet, da er hochperformant ist und eine große Auswahl an Plug-ins hat. Zur Unterstützung des Projektes wurden die folgenden Plug-ins verwendet:

- *Python*: Grundplugin zur Unterstützung von Python in VS Code
- *Pylance*: Erweiterter Sprachsupport, z.B. farbliche Hervorhebung von Code, Überprüfung von Datentypen, Auto-Vervollständigung von Variablen- und Funktionsnamen, etc.
- *Ruff*: Fehlerüberprüfung und Code-Formatierung

5.2 Zielplattform

Die Software wurde basierend auf dem Beeware-Toolkit in Python erstellt. Die Programmiersprache Python wurde gewählt, da sie sehr flexibel und mächtig ist. Durch eine große Auswahl an Standardbibliotheken sind neue Features schnell und kostengünstig umzusetzen. Eine hohe Lesbarkeit des Codes erleichtert die Wartung.

Beeware ist eine Sammlung an Werkzeugen, die es ermöglicht, in Python Plattformunabhängige Benutzeranwendungen zu erstellen. Ein Fokus von Beeware liegt darin, die nativen Bedienelemente der jeweiligen Betriebssysteme zu verwenden. Das führt zu sehr performanten Anwendungen. Die wichtigsten Werkzeuge aus Beeware, die in dem Projekt zum Einsatz kamen, sind:

- *Briefcase*: Ein Tool, das das Erstellen von Beeware-Projekten ermöglicht und diese zu nativen Mac-, Windows-, Linux-, iOS- und Android-Anwendungen verpacken kann.
- *Toga*: Ein Frontend-Framework, das darauf spezialisiert ist, auf verschiedenen Plattformen native Bedienelemente zu verwenden. Der Aufbau von Benutzeroberflächen ist an CSS angelehnt.

Zur Sicherstellung der Qualität sowie um im Kostenrahmen zu bleiben, wurde zunächst auf nur ein Ziel-Betriebssystem zugearbeitet. Aufgrund der bereits vorhandenen Windows-Computer im Coffee-Shop ist die Entscheidung für Windows gefallen. Im Rahmen von Folgeprojekten kann das Programm jedoch mit relativ geringem Aufwand auch für Mac, Android und iOS zur Verfügung gestellt werden.



5.3 Entwicklung

5.3.1 Projekterstellung

Zunächst wurde die Installation von Python überprüft und die Version (3.12) bestätigt. Anschließend wurde ein Virtual Environment angelegt und aktiviert. In diesem wurde mithilfe des Python-Package-Managers *pip* das Briefcase-Modul installiert.

Anschließend wurde mithilfe des Befehls *briefcase new* ein neues Projekt angelegt. Dadurch wurde automatisch die Projektstruktur sowie die grundlegenden Daten angelegt.

```
PS>briefcase new

Let's build a new Briefcase app!

-- Formal Name -----

First, we need a formal name for your application.

This is the name that will be displayed to humans whenever the name of the
application is displayed. It can have spaces and punctuation if you like, and
any capitalization will be used as you type it.

Formal Name [Hello World]: Coffee Helper
```

Auszug aus dem „*briefcase new*“ Befehl

Nach Erstellen des Projekts wurden mithilfe des Befehls „*briefcase dev*“ die benötigten Python-Module installiert sowie wie die Standard-GUI getestet.

Nachdem die Funktionalität bestätigt wurde, wurde ein Git-Repository initialisiert. Es wurde ein erster Commit durchgeführt, der die Standard-Daten, die von Briefcase erstellt wurden, enthält.

Anschließend wurde ein privates Repository auf GitHub angelegt, als Remote eingerichtet, und der aktuelle Stand gepusht. In der fortlaufenden Entwicklung wurden regelmäßig Commits angelegt und zur Remote gepusht.

5.3.2 Erstellung der GUI

Der grundlegende Aufbau des Programms besteht aus Klassen, die von der *toga.Box*-Klasse erben. Die *toga.Box* ist ein Container, der sich ähnlich wie eine CSS-Flexbox verhält. In diesen können beliebige Benutzeroberflächen-Objekte eingefügt werden.

Der Kern des Programms ist die *MainContainer*-Klasse. Sie erbt ebenfalls von der *toga.Box*-Klasse, ist jedoch um diverse Methoden erweitert, die den Inhalt der Box steuern und Events von Bedienelementen abfangen.



Die jeweiligen Schritte des Programms sind in der `screens.py`-Datei untergebracht. Für jedes Layout wurde eine Klasse erstellt, die nach Bedarf vom `MainContainer` initialisiert und in die Benutzeroberfläche eingefügt wird. Einige dieser Klassen erben von der selbst erstellten `ImageTextBox`-Klasse, die ein Bild oben und darunter einen Text anzeigen kann. Dieses Layout wurde häufig verwendet, daher hat es sich angeboten, dies zu vereinheitlichen.

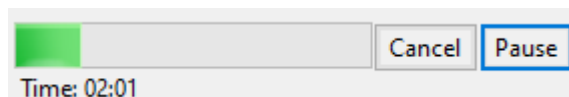
```
class ImageTextBox(toga.Box):
    """Base class for creating a box with an image at the top and some text at
    the bottom."""

    def __init__(self):
        super().__init__(style=Pack(flex=1, direction=COLUMN, padding=(5, 0)))
        self.image_box = toga.Box(style=Pack(direction=COLUMN, flex=1,
padding=5))
        self.label_box = toga.Box(style=Pack(direction=COLUMN, padding=(5,
0)))
        self.add(self.image_box, self.label_box)
```

Die `ImageTextBox`-Klasse

5.3.2.1 Das Timer-Element

Da einige Schritte in der Zubereitung von Kaffee eine gewisse Wartezeit enthalten, wurde eine Möglichkeit gesucht, diese zu messen und grafisch aufzubereiten. Daher wurde ein Timer-Modul entwickelt, das einfach mit Übergabe der gewünschten Zeit diese misst und anzeigt.



Eine Herausforderung hierbei war, die Zeit präzise zu messen und dabei nicht das Programm zu blockieren. Dies wurde umgesetzt, indem, wenn der Timer gestartet wird, die aktuelle Zeit (in Sekunden seit 01.01.1970, die sog. Unix Time) in einer Variable gespeichert wird. Dann wird regelmäßig – zweimal pro Sekunde – erneut die aktuelle Zeit aufgezeichnet. Aus der Differenz der beiden Werte ergibt sich die seit Start des Timers vergangene Zeit. Diese wird dann verwendet, um die Fortschrittsanzeige zu aktualisieren und ggf. den Timer zu beenden, nachdem er null oder negative Sekunden erreicht hat.

Die halbsekündigen Prüfungen werden in einer asynchronen Methode durchgeführt. Diese enthält eine Schleife, die sich wiederholt, solange der Timer läuft. Nachdem eine Aktualisierung des Timers durchgeführt wurde, wird die `asyncio.sleep`-Methode ausgeführt. Diese gibt die Kontrolle an das Programm zurück, das in der Zeit andere asynchrone Prozesse, z.B. GUI-Elemente laden oder Button-Klicks verarbeiten, abarbeiten kann. Nach der vorgegebenen Zeit (hier 0,5 Sekunden) kehrt das Programm zu der Stelle zurück, wo es aufgehört hat.



```
async def tick_timer(self, app):
    while self.is_running:
        delta = time.time() - self.start_time
        if delta >= self.duration:
            self.finish()
            return

        self.update_ui(delta)
        # asynchronous sleep to prevent blocking the UI
        await asyncio.sleep(0.5)
```

Auszug aus der Timer-Klasse

5.3.3 Geschäftslogik

Im Folgenden wird intensiver auf die Entwicklung und Funktionsweise der Geschäftslogik eingegangen.

5.3.3.1 Auslesen der Zubereitungsschritte

Die Zubereitungsmethoden werden in JSON-Dateien vorgegeben, die im Abschnitt [Dateiformat für die Zubereitungsmethode](#) detaillierter erläutert werden. Zum Auslesen dieser Methode wurde die `JsonReader`-Klasse entwickelt. Diese hat zwei Methoden:

- `read_steps`: Prüft das `resources/instructions`-Verzeichnis. Wenn es Dateien enthält, werden die, die auf „.json“ enden, mit Error Handling ausgelesen. Anschließend wird mit der `verify_valid_instructions`-Methode geprüft, ob die Dateien das richtige Format haben, um als Zubereitungsanweisung gelesen zu werden. Alle validen Datensätze werden als Liste zurückgegeben.
- `verify_valid_instructions`: Führt eine oberflächliche Prüfung der Daten in den ausgelesenen Dictionaries durch. Prüft, ob die notwendigen Daten vorhanden sind und ob sie den richtigen Datentyp haben.

Beim Programmstart wird die `read_steps`-Methode vom `MainContainer` aufgerufen, um die Anweisungen auszulesen. Anschließend werden die Optionen dem Nutzer als Dropdown-Menü angezeigt. Wenn der Nutzer seine Auswahl bestätigt, werden die Zubereitungsanweisungen in ein `Recipe`-Objekt geladen.

5.3.3.2 Die Recipe-Klasse

Die `Recipe`-Klasse ist ein Singleton-Objekt, das das aktuell ausgewählte Rezept repräsentiert. Die Entscheidung für ein Singleton-Objekt wurde getroffen, damit verschiedene Komponenten der Applikation den aktuellen Fortschritt der Zubereitung überprüfen und ggf. anpassen können.

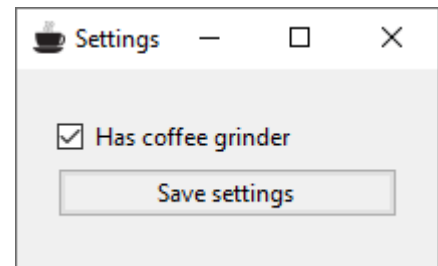
Mit der `load_recipe`-Methode wird ein Rezept aus einem übergebenen Dictionary ausgelesen. Anschließend kann mit `next_step` und `previous_step` der Fortschritt der Zubereitung gesteuert werden. `get_current_step` gibt den aktuellen Schritt als Dictionary aus, oder `None`, wenn das Ende erreicht wurde.



5.3.3.3 Einstellungen

Die Anwendung verwaltet Einstellungen in einer *config.ini*-Datei, die vom Python-Standard-Modul configparser verwaltet wird. Hierzu ist die Klasse *Settings* angelegt worden, die zum Programmstart die aktuellen Einstellungen ausliest, sofern eine Einstellungsdatei vorhanden ist. Sie stellt ebenfalls Funktionen zur Verfügung, die das Schreiben der Einstellungen ermöglichen.

Die Einstellungsdatei wird verwendet, um zu speichern, ob der Nutzer eine Kaffeemühle besitzt. Sollte keine Einstellungsdatei vorhanden sein oder jene keine Informationen zur Kaffeemühle enthalten, wird der Nutzer bei der ersten Zubereitung gefragt, ob er eine Kaffeemühle besitzt. Zudem gibt es über das Menüband die Option, ein Einstellungsfenster zu öffnen, indem er diese Einstellung ändern kann. In beiden Fällen wird die Einstellung anschließend in der *config.ini* gespeichert.



5.3.3.4 Berechnung des Verhältnisses von Wasser und Kaffee

Nachdem der Nutzer eine Zubereitungsmethode gewählt hat, kann er die Menge an Kaffee/Wasser anpassen. Hierbei wird das Verhältnis zwischen Kaffee und Wasser immer beibehalten. Hierzu wurden einige Utility-Funktionen im Modul *calc.py* implementiert.

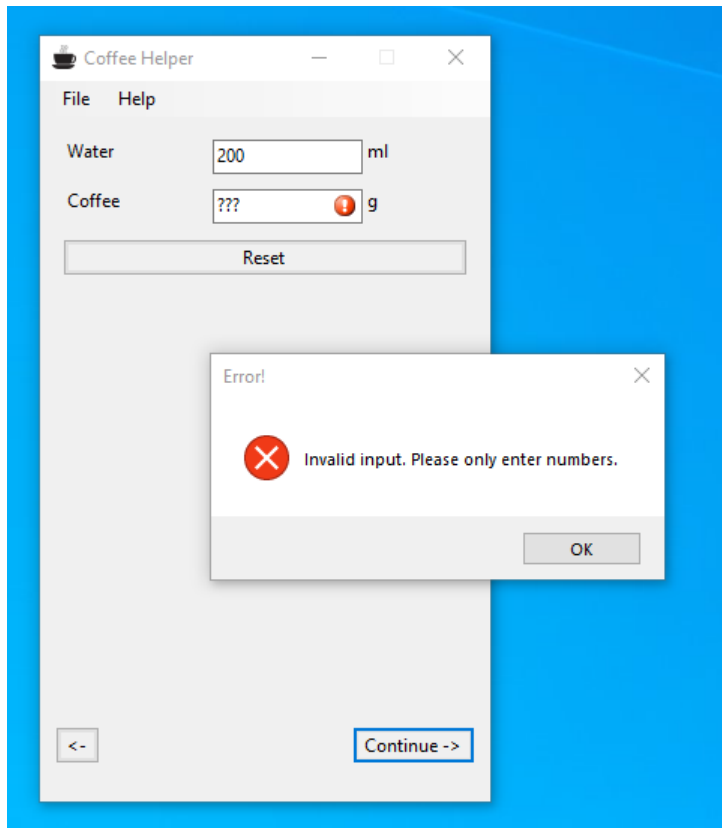
```
def calculate_ratio(coffee: int, water: int) -> float:
    return coffee / water

def calculate_coffee_from_water(water: int, ratio: float) -> float:
    return water * ratio

def calculate_water_from_coffee(coffee: int, ratio: float) -> float:
    return coffee / ratio
```

Auszug aus *calcs.py*

Die Eingabefelder überprüfen, ob das Input eine gültige Zahl darstellt. Wenn nicht, wird im Textfeld „???“ dargestellt und das Fortschreiten zum nächsten Schritt wird mit einem Fehler-Dialogfenster verhindert.



5.3.4 Dateiformat für Zubereitungsmethode

Die Zubereitungsmethoden wurden als JSON angelegt. Im Folgenden ist der Aufbau der Dateien beschrieben:

- *name*: Der Name des Rezepts, der im Auswahlménü der Anwendung angezeigt wird.
- *image_folder* (optional): Der Name des Ordners in *resources/images*, indem sich die Bilder für die Anweisungen befinden.
- *grind* (optional): Der empfohlene Mahlgrad.
- *ratio*:
 - *water*: Die empfohlene Menge an Wasser
 - *coffee*: Die empfohlene Menge an Kaffee
- *steps*: Liste der einzelnen Bearbeitungsschritte mit folgenden Unterpunkten:
 - *text*: Die Textbeschreibung des Arbeitsschritts
 - *image* (optional): der Dateiname des Bildes im *image_folder*
 - *timer* (optional): Die Dauer des Timers für diesen Schritt, falls benötigt.

In der Auslieferungsversion der Anwendung sind Zubereitungsmethoden für die Aeropress und die French Press enthalten. In zukünftigen Updates kann sie leicht um weitere Anweisungen erweitert und die vorhandenen bearbeitet werden.

5.3.5 Packaging

Nach Abschluss der Entwicklung wurde die Anwendung zum Installer für Windows kompiliert. Hierfür wurde abermals Briefcase eingesetzt. Dies wurde mit dem Befehl



briefcase package windows durchgeführt. Dies erzeugt eine MSI-Installationsdatei mithilfe des WiX-Toolsets. Diese lässt sich ohne Probleme auf den meisten Windows-Maschinen installieren.

5.3.6 Testing

Das Programm wurde zur Qualitätssicherung ausgiebig getestet. Zum Prüfen der Berechnungs-Funktionen wurden Unittests erstellt. Die Testfunktionalität wird von Briefcase basierend auf dem Python Unittest-Modul `pytest` zur Verfügung gestellt.

```
def test_ratio_1_1():
    assert calcs.calculate_ratio(1, 1) == 1.0

def test_ratio_50_10():
    assert calcs.calculate_ratio(50, 10) == 5.0

def test_ratio_10_50():
    assert calcs.calculate_ratio(10, 50) == 0.2

def test_ratio_0_0():
    """Testing against zero division errors."""
    assert calcs.calculate_ratio(0, 0) == 0
```

Auszug aus Testklasse

Zudem wurde nach Abschluss der Hauptentwicklungsphase eine explorative Testphase durchgeführt. Hierbei hat der Autor die Funktionalität des Programmes ausgiebig manuell getestet und hierbei Protokoll geführt. Ein [Auszug aus dem Protokoll](#) ist der Dokumentation angehängt.



6 Abnahme- und Einführungsphase

6.1 Abnahme durch den Fachbereich

Die Übergabe wurde zusammen mit Herrn Bohne der Simply Coffee GmbH durchgeführt.

Zunächst wurde die erstellte Installationsdatei übergeben. Das Programm wurde installiert und dessen Funktionalität bestätigt. Herr Bohne hat eine kurze Einführung in die Bedienung der Software bekommen.

Anhand des Pflichtenhefts wurden die Projektanforderungen geprüft und die Umsetzung dieser bestätigt.

Darüber hinaus wurde die Kundendokumentation übergeben.

6.2 Deployment und Einführung

Zunächst wurde auf der Webseite der Simply Coffee GmbH eine Download-Seite erstellt. Ein Link zu dieser wurde auf der Homepage hinzugefügt. Auf der Download-Seite wurde ein Link zum Herunterladen der Installationsdatei hinzugefügt. Darüber hinaus wurde ein Ausschnitt aus der Kundendokumentation, der die Installation erläutert, auf dieser Seite eingefügt.

Anschließend wurde ein QR-Code generiert, der auf die Download-Seite zeigt. Dieser wurde von der Marketing-Abteilung in diverse Werbeflächen integriert, z.B. Schilder im Laden und Sticker auf den Kaffeeprodukten. Dieser soll die Endkunden dazu anreizen, das Programm herunterzuladen.

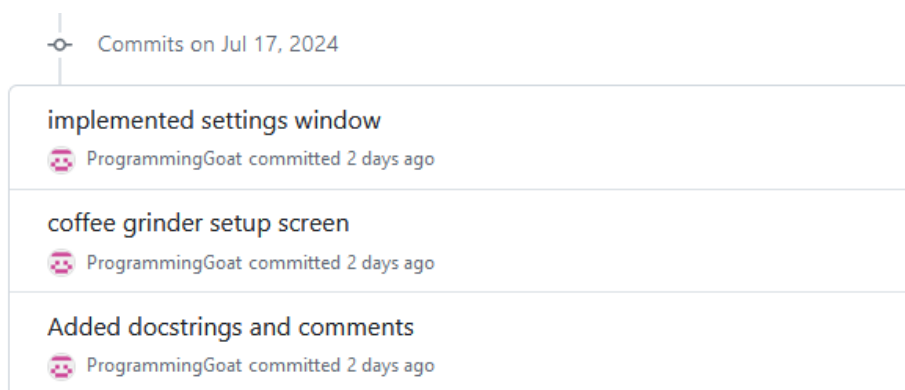
Insgesamt lief das Deployment wie erwartet und ohne Probleme.



7 Dokumentation

Im Laufe des Entwicklungsprozesses wurde kontinuierlich und detailliert dokumentiert. Es wurde eine fortlaufende Tagesdokumentation geführt, in der jeder vollzogene Arbeitsschritt formfrei notiert wurde. Diese wurde dann im Rahmen der formalen Dokumentation in die vorliegende Form überführt.

Zudem wurde jeder logische Entwicklungsschritt in Form von Commits ins Git-Repository geschrieben. So sind Änderungen eindeutig nachvollziehbar.

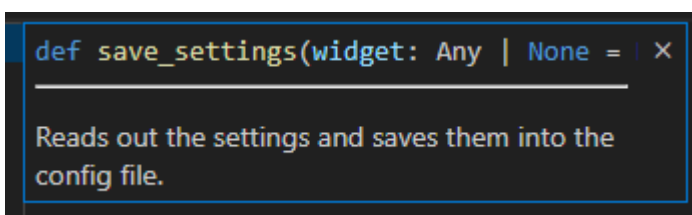


Ein Auszug aus der Commit History.

Der Code wurde durch sogenannte DocStrings ergänzt. Sie stellen eine Möglichkeit dar, Dokumentation von Modulen, Klassen und Methoden in den Code zu integrieren. Sie wird von den meisten IDEs automatisch erkannt und bspw. bei Eingabe des Klassennamens angezeigt. Sie stellen eine starke Grundlage der In-Code-Dokumentation dar und wurden, wo weitere Erläuterung sinnvoll war, durch Kommentare ergänzt.

```
def save_settings(self, widget=None):
    """Reads out the settings and saves them into the config file."""
    has_grinder = self.grinder_checkbox.value
    settings.set_coffee_grinder(has_grinder)
    self.close()
```

Der DocString der einer Methode



VS Code zeigt den DocString der Methode an.

Zudem wurde eine Kundendokumentation erstellt, die die Bedienung der Software einfach erklärt. Diese wurde der Dokumentation angehängt.



8 Fazit

8.1 Soll-/Ist-Vergleich

Das Projekt wurde allen Anforderungen entsprechend umgesetzt. Die GUI ist benutzerfreundlich und funktionsfähig. Zubereitungsschritte für Aeropress und French Press wurden implementiert. Bei Eingabe von Wassermenge kann Kaffeemenge berechnet werden und umgekehrt. Die Software führt den Nutzer Schritt für Schritt durch die Zubereitung. Timer für Wartezeiten sind implementiert. Die Einstellung, ob man eine Kaffeemühle hat, ist vorhanden und persistent.

Die Zeitplanung wurde weitestgehend eingehalten. Die Testphase hat eine Stunde mehr in Anspruch genommen als geplant. Diese Zeit konnte jedoch in der Erstellung der GUI eingespart werden. Somit konnte der zeitliche Rahmen exakt eingehalten werden.

Projektphase	Geplante Zeit	Tatsächliche Zeit	Differenz
Kundengespräche	2 h	2 h	-
Planung	8 h	8 h	-
Analyse	6 h	6 h	-
Implementierung	30 h	30 h	-
Erstellen des Projekts in Beeware	1 h	1 h	-
Erstellen der GUI	12 h	11 h	-1 h
Implementierung der Geschäftslogik	11 h	11 h	-
Testing / Debugging	5 h	6 h	+1 h
Building / Packaging	1 h	1 h	-
Erstellen der Dokumentation	28 h	28 h	-
Projektdokumentation	25 h	25 h	-
Kundendokumentation	2 h	2 h	-
Quellcodedokumentation	1 h	1 h	-
Kundenübergabe	2 h	2 h	-
Soll- / Ist-Vergleich	4 h	4 h	-
Summe	80 h	80 h	-

8.2 Lessons Learned

Im Zuge des Projektes hat der Autor gelernt, wie man mit einem linearen Entwicklungsprozess wie dem Wasserfallmodell ein Projekt umsetzt. Hierbei konnte er die Vorteile dieses hochstrukturierten Vorgangs in der Praxis erleben.

Darüber hinaus hat er tiefere Erfahrungen im Software-Testing gesammelt. Er hat gelernt, wie man automatisiertes Testing in den Entwicklungsprozess integriert. Darüber hinaus hat er ein besseres Verständnis für den Arbeitsaufwand von Testing und Bugfixes gewonnen.



8.3 Ausblick

Nachdem das Projekt deployt und beworben wurde, soll nun abgewartet werden, welches Feedback von den Endkunden kommt. Sollte es zu Bugs oder Problemen kommen, werden diese als Teil des Serviceumfangs kostenfrei behoben.

In einem zeitnahen kleinen Anschlussprojekt soll das Firmenbranding in die Anwendung integriert werden.

Sollte das Programm gut aufgenommen werden, werden der Autor und die Simply Coffee GmbH erneut zusammenkommen, um die Entwicklung für andere Betriebssysteme zu beginnen. Da Beeware plattformübergreifend ist, sollte dies mit nur geringfügigen Anpassungen möglich sein.



9 Quellen

- <https://docs.beeware.org/>
- <https://toga.readthedocs.io/>
- <https://docs.python.org/>
- <https://git-scm.com/doc>
- <https://pencil.evolus.vn/>
- <https://www.w3schools.com/python/>
- <https://it-berufe-podcast.de/>
- <https://stackoverflow.com/>
- <https://oer-informatik.de/uml-klassendiagramm-assoziaton>
- <https://www.ionos.de/digitalguide/websites/web-entwicklung/anwendungsfalldiagramm/>
- <https://tree.nathanfriend.io/> (zur Darstellung der Ordnerstruktur)
- <https://de.wikipedia.org/> (für Begriffsdefinitionen)
- <https://openclipart.org/> (diverse Bilder, alle im Public Domain)
- Zubereitungs-Bilder in der App – Screenshots von:
 - <https://www.youtube.com/watch?v=rzoR82KsK5A> (FP)
 - https://www.youtube.com/watch?v=j6VLT_jUVPc (AP)



10 Glossar

Agile Softwareentwicklung	Ansätze im Softwareentwicklungsprozess, die die Transparenz und Veränderungsgeschwindigkeit erhöhen sollen, z.B. Scrum, Extreme Programming
Android	Open-Source Betriebssystem für Smartphones auf der Basis von Linux.
Beeware	Ein Framework basierend auf Python zum Entwickeln von Programmen, die auf mehreren Betriebssystemen funktionieren.
Betriebssystem	Die Grundliegende Software, die die Hardware eines Computers verwaltet und das Ausführen von Programmen ermöglicht. Bspw. Windows, Mac.
Briefcase	Ein Werkzeug, das Teil des Beeware-Toolkits ist. Es wird zum Erstellen der Projektdaten sowie dem Kompilieren und Verpacken zu Betriebssystem-Nativen Anwendungen verwendet.
Button	Eine klickbare Schaltfläche in der GUI.
CSS	Cascading Style Sheets (eng. „gestufte Gestaltungsbögen“) ist eine Stylesheet-Sprache, die die Gestaltung von HTML-Seiten ermöglicht. Sie ist neben HTML und JavaScript einer der Kernsprachen des World Wide Webs.
GB	Gigabyte, eine Maßeinheit von Daten in der Informatik. Entspricht 10^9 Byte.
Git	Freie Software zur Versionsverwaltung von Dateien. Ermöglicht es, Veränderungen im Code über verschiedene Versionen aufzuzeichnen, sowie ggf. gemeinsam an einem Quellcode zu arbeiten.
Git – Commit	Ein Commit ist eine Änderung, die im Git Repository dokumentiert wird.
Git – Push	Mit dem Push-Befehl können die aktuellen Änderungen des lokalen Git Repository auf die Remote übertragen werden.
Git – Remote	Git Repositories können auf externen Plattformen wie z.B. GitHub hochgeladen werden. Auf diese kann die lokale Installation von Git dann zugreifen und Veränderungen hoch- und herunterladen.
Git – Repository	Das Git Repository ist der Ordner bzw. die Dateien, die die Versionsgeschichte aufzeichnen und von Git verwaltet werden.
GUI	Abk. von englisch „graphical user interface“. Grafische Benutzerschnittstelle oder Benutzeroberfläche. Sie ist die Bedienoberfläche des Programms, mit dem der Endnutzer z.B. mit Maus, Tastatur oder Touchscreen das Programm



	bedient.
Hardware	Der physikalische Anteil eines Computers, z.B. Mainboard, Arbeitsspeicher, CD-Laufwerk, etc., das Gegenstück zu Software.
INI-Datei	Von engl. „initialization“ (Initialisierung), eine Art von menschenlesbarer Konfigurationsdatei.
iOS	Betriebssystem für Smartphones des amerikanischen Herstellers Apple.
JSON	„JavaScript Object Notation“, Format zum Speichern von Daten als Textdatei, ursprünglich für die Programmiersprache JavaScript entwickelt, mittlerweile weit verbreitet.
Kompilierung	Umwandlung von Programmiersprachen in eine Zielsprache, üblicherweise Assembly oder Maschinencode.
MSI	.msi ist die Dateierweiterung für Windows-Installationsdateien.
nativ	(lat. <i>nativus</i> = „natürlich“) hier: vom Betriebssystem bereitgestellt, z.B. in Windows die Standard-Buttons, Textfelder und andere Bedienelemente.
PAP	Programm-Ablauf-Plan, ein Diagramm, das den Ablauf von einem Programm visuell darstellt.
persistent	Daten, die, auch wenn das Programm geschlossen wird, vorhanden bleiben, sind persistent. Dies wird z.B. durch Speichern in einer Datei bewerkstelligt.
pip	pip ist das empfohlene Paketverwaltungsprogramm für die Programmiersprache Python. Es kann Pakete aus dem Python Package Index (PyPI) beziehen, installieren und verwalten.
Plug-in	Optionale Komponente, die eine bestehende Software erweitert.
Python	Eine flexible Programmiersprache, die objektorientierte Programmierung unterstützt.
QR-Code	„Quick-Response-Code“, ein zweidimensionaler Code, der mit einem Lesegerät oder einer Kamera ausgelesen werden kann, ähnlich einem Barcode
RAM	Random-Access Memory (engl. „Speicher mit wahlfreiem Zugriff“), eine Form des Datenspeichers die üblicherweise als Arbeitsspeicher in Computern verwendet wird.
Singleton	Ein Design-Pattern (wiederverwendbare Lösung für eine häufige Problemstellung in der Programmierung), das nur eine Instanz eines Objektes zulässt. Oft für den globalen Zugriff auf ein Objekt in einer gesamten Applikation verwendet.



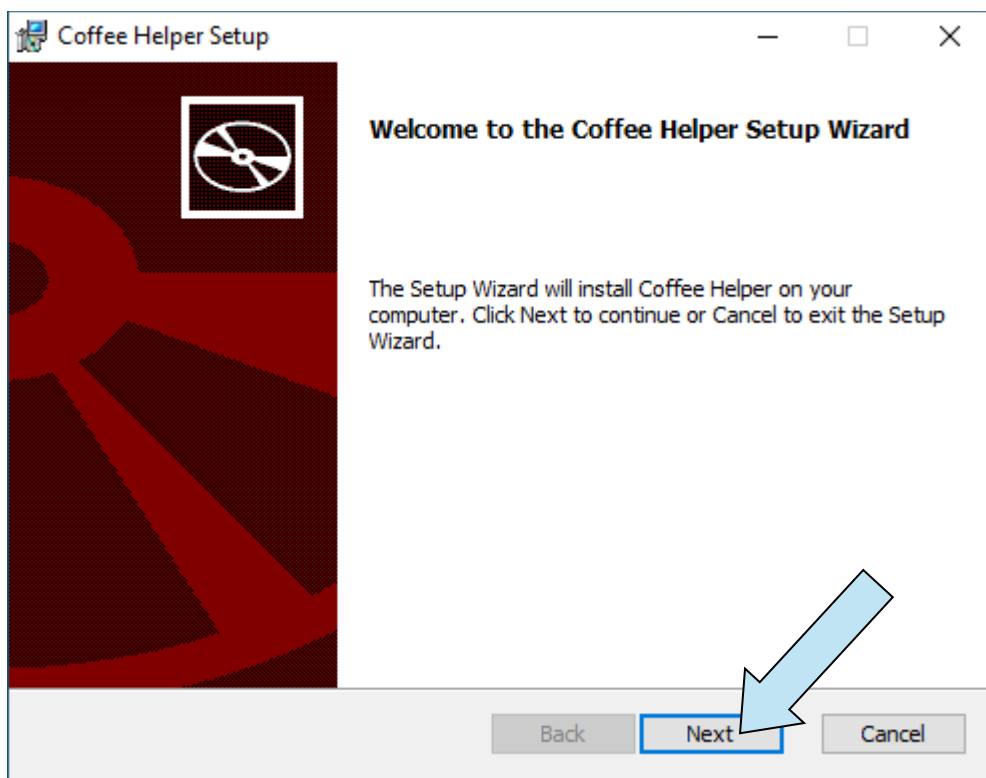
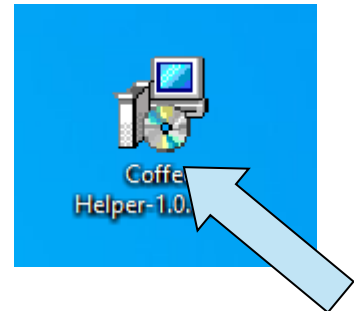
Software	Computerprogramme und die zugehörigen Daten, das Gegenstück zu Hardware.
Toga	Ein Framework zur Erstellung von Benutzeroberflächen in Python, in Zusammenhang mit Beeware.
UML	Unified Modeling Language (vereinheitlichte Modellierungssprache), eine Spezifikation zur visuellen Darstellung von Software-Teilen und anderen Systemen. Bezieht sich oft aufs UML-Klassendiagramm.
Unittest	Softwaretest, bei dem einzelne, abgrenzbare Teile von Computerprogrammen (z.B. Module, Unterprogramme, Klassen) überprüft werden. Oft automatisiert.
Virtual Environment	(auch virtualenv oder venv) ein Ordner, indem eine eigene Python-Installation angelegt wird, in der, ohne die Hauptinstallation zu beeinflussen, Module installiert werden können. Dieses wird durch Aufrufen eines Skripts aktiviert und ersetzt dann alle Aufrufe des globalen Python-Interpreters.
VS Code	Visual Studio Code, ein kostenloser, quelloffener, ursprünglich von Microsoft entwickelter Quelltexteditor.
Windows	Ein vom Software-Unternehmen Microsoft entwickeltes Betriebssystem.



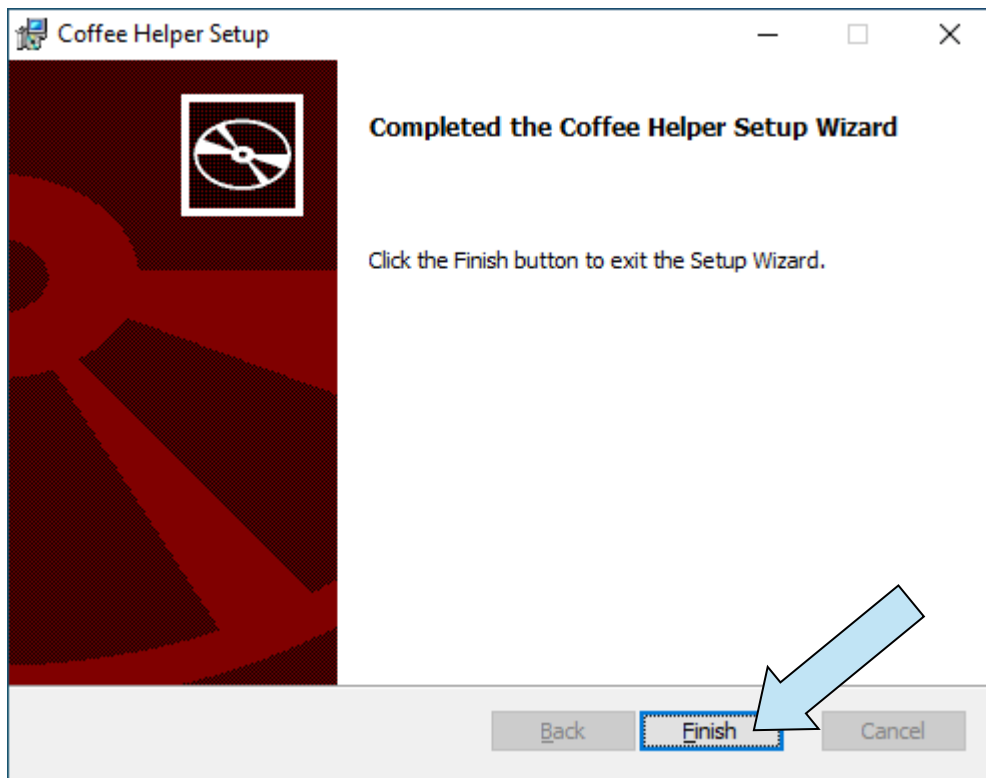
11 Kundendokumentation

11.1 Installation

Um die Installation zu beginnen, doppelklicken Sie die Installationsdatei, die Sie heruntergeladen haben.



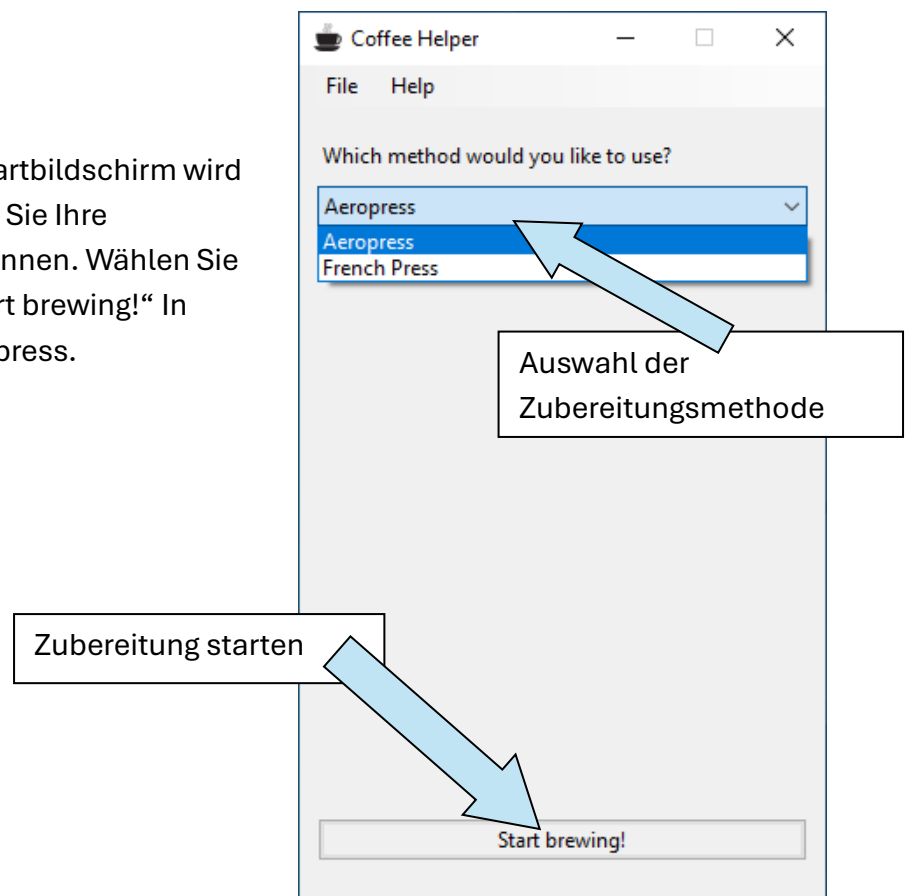
In dem Installationsfenster können Sie nun immer wieder auf „Next“ klicken. Wenn Sie sich bei den Optionen nicht sicher sind, lassen Sie einfach alles bei den Standardeinstellungen.



Wenn Sie dieses Fenster sehen, ist die Installation abgeschlossen.

11.2 Bedienung

Starten Sie das Programm. Im Startbildschirm wird Ihnen ein Menü angezeigt, indem Sie Ihre Zubereitungsmethode wählen können. Wählen Sie eine aus und klicken Sie auf „Start brewing!“ In diesem Beispiel wählen wir Aeropress.



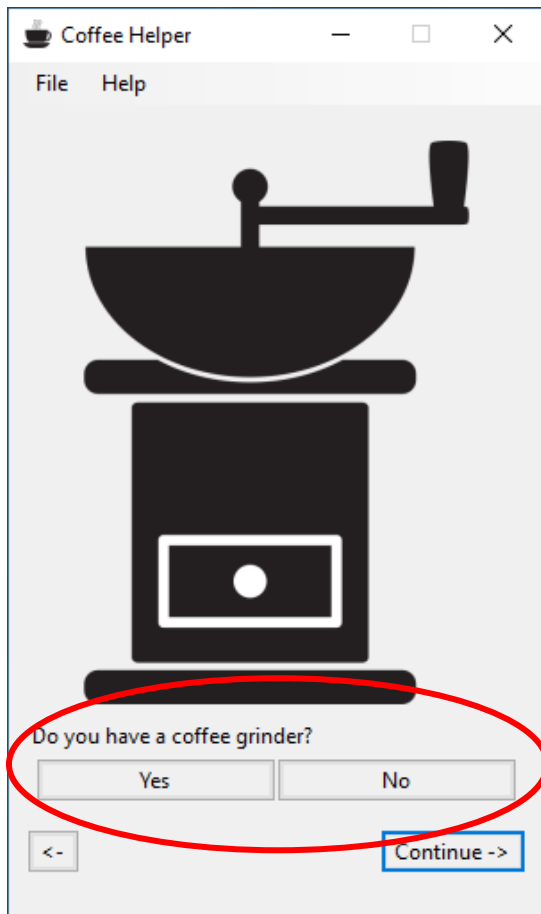


The screenshot shows the 'Coffee Helper' application interface. At the top, there are two input fields: 'Water' with the value '200 ml' and 'Coffee' with the value '11 g'. Below these fields is a 'Reset' button. At the bottom, there are two buttons: '<- Vorheriger Schritt' and 'Nächster Schritt -> Continue'. Annotations with arrows point to the 'Coffee' input field, the 'Reset' button, and the '<- Vorheriger Schritt' button. The text 'Kaffeeeingabe' is placed near the 'Coffee' input field, 'Wassereingabe' is placed near the 'Water' input field, and 'Zurücksetzen' is placed near the 'Reset' button.

In der nächsten Ansicht können Sie eingeben, wie viel Kaffee Sie zubereiten möchten. Sie können entweder die Menge an Wasser oder Kaffeebohnen bzw. -pulver eingeben. Der jeweils andere Wert wird dann automatisch errechnet. Mit dem „Reset“-Button können Sie die Mengen auf ihre ursprünglichen Werte zurücksetzen. Bitte geben Sie in die Textfelder nur Zahlenwerte ein. Die Anwendung verwendet das englische Zahlenformat. Verwenden Sie daher bitte Punkte statt Kommas, um Dezimalstellen zu trennen.

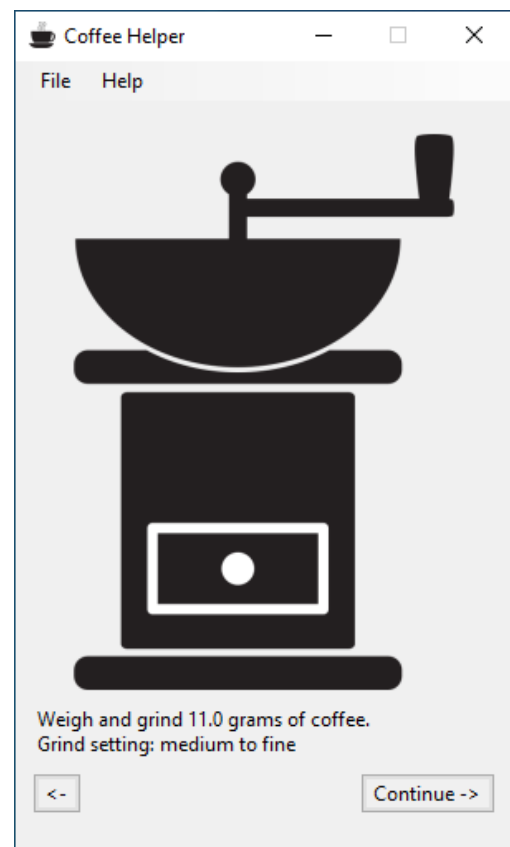
Im unteren Bereich des Fensters finden sie zwei Schaltflächen. Mit einem Klick auf „<-“ gelangen sie zum vorherigen Schritt zurück. Mit „Continue ->“ fahren Sie zum nächsten Schritt fort.

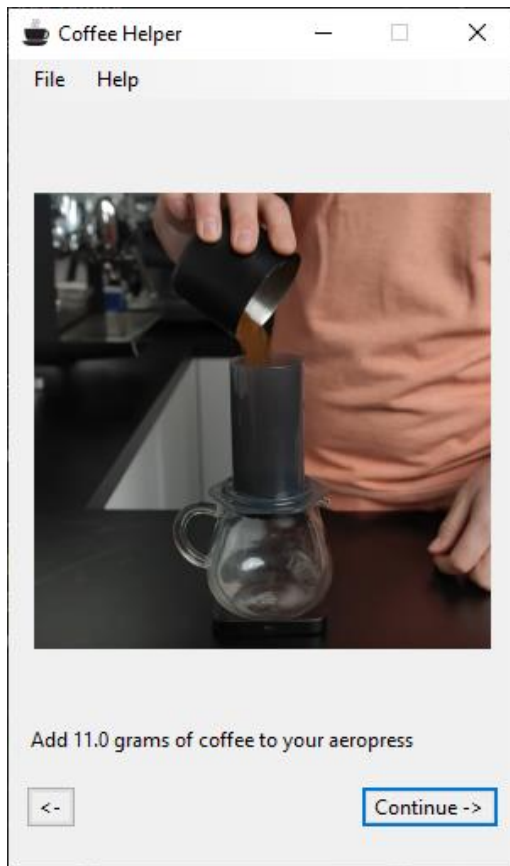
Nachdem Sie die Kaffeemenge eingegeben haben, klicken Sie auf „Continue ->“.



Sollten Sie die Anwendung zum ersten Mal einsetzen, wird Ihnen nun diese Abfrage angezeigt. Bitte wählen Sie aus, ob sie eine Kaffeemühle besitzen oder nicht. Diese Einstellung wird dauerhaft gespeichert. Wenn Sie dies ändern möchten, können Sie dies in den Einstellungen tun. In diesem Beispiel wählen wir „Yes“ aus.

Folgen Sie nun den Anweisungen auf dem Bildschirm und wiegen Sie die angegebene Menge an Kaffee hab. Falls Sie eine Kaffeemühle besitzen und lose Bohnen verwenden, mahlen Sie diese jetzt.

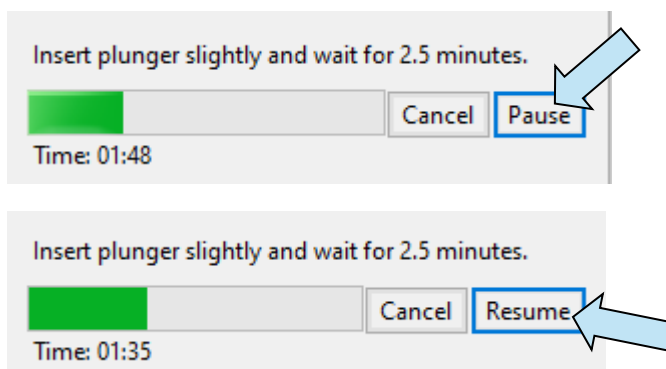




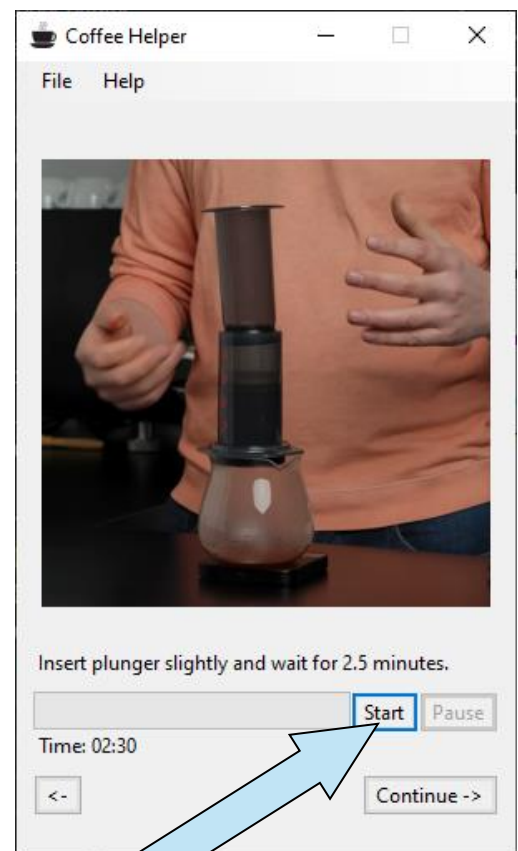
Die Anwendung führt Sie nun Schritt für Schritt durch die Zubereitung. Folgen Sie jeweils den Anweisungen und klicken Sie anschließend auf „Continue ->“.

Sollte ein Schritt einen Timer enthalten, klicken Sie auf „Start“ und warten Sie. Wenn der Timer abgelaufen ist, gibt das Programm einen Ton von sich.

Sie können den Timer jederzeit mit einem Klick auf „Pause“ pausieren. Anschließend können Sie ihn mit einem Klick auf „Resume“ an der gleichen Stelle fortsetzen.



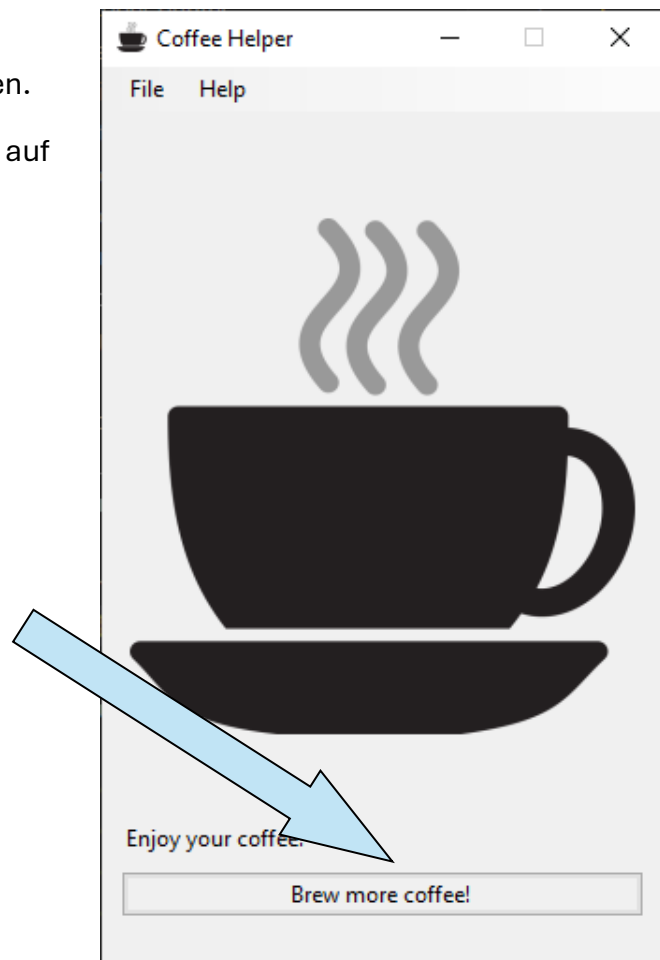
Mit einem Klick auf „Cancel“ wird der Timer abgebrochen und zurückgesetzt. Anschließend können Sie ihn mit einem Klick auf „Start“ erneut starten.





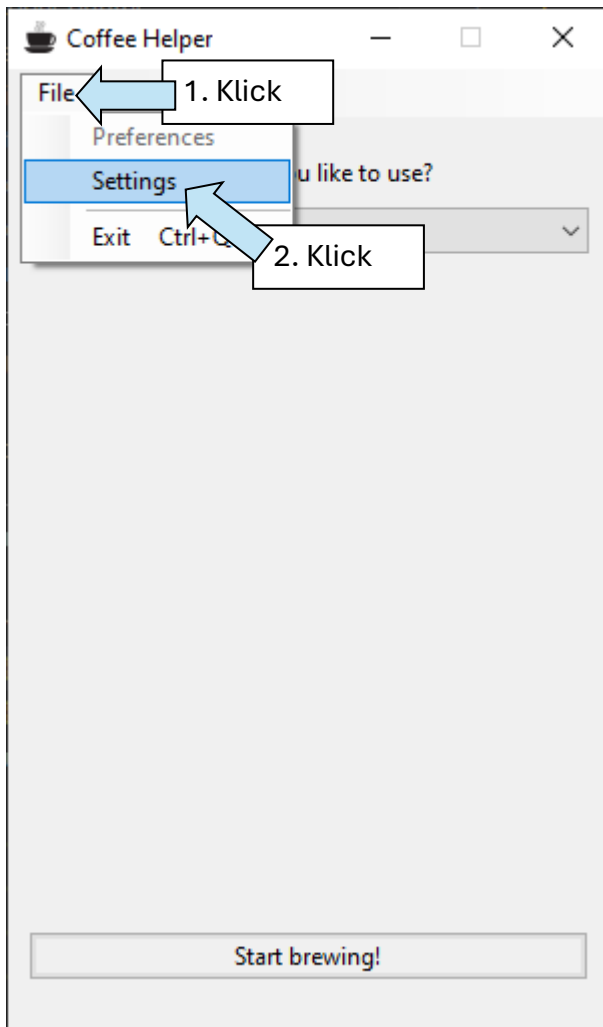
Wenn Sie diese Anzeige sehen, ist die Zubereitung Ihres Kaffees abgeschlossen.

Sie können den Prozess mit einem Klick auf „Brew more coffee!“ erneut starten.



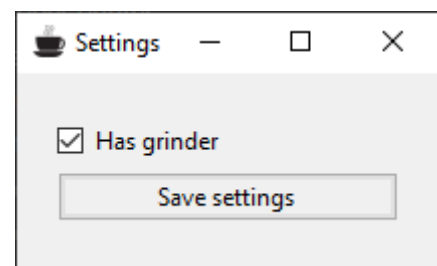


11.3 Einstellungen



Klicken Sie zunächst auf File, dann auf Settings, um auf das Einstellungsfenster zuzugreifen.

Es öffnet sich dieses Fenster. Mit einem Klick auf das kleine Rechteck können Sie auswählen, ob Sie eine Kaffeemühle haben oder nicht. Die Einstellungen werden übernommen, indem Sie auf „Save settings“ klicken.





12 Auszug aus dem Testprotokoll

12.1 Explorative Testing

12.1.1 Standardnutzungsfall

Programm ausgeführt, alle Standardoptionen ausgewählt gelassen.

Problem:

- Timer versucht zu klingeln, nachdem der Schritt übersprungen wurde
 - o Fehlermeldung: `AttributeError: 'NoneType' object has no attribute 'beep'`

12.1.2 Ungewöhnliche Eingabewerte

Bei Kaffee-/Wassereingabe:

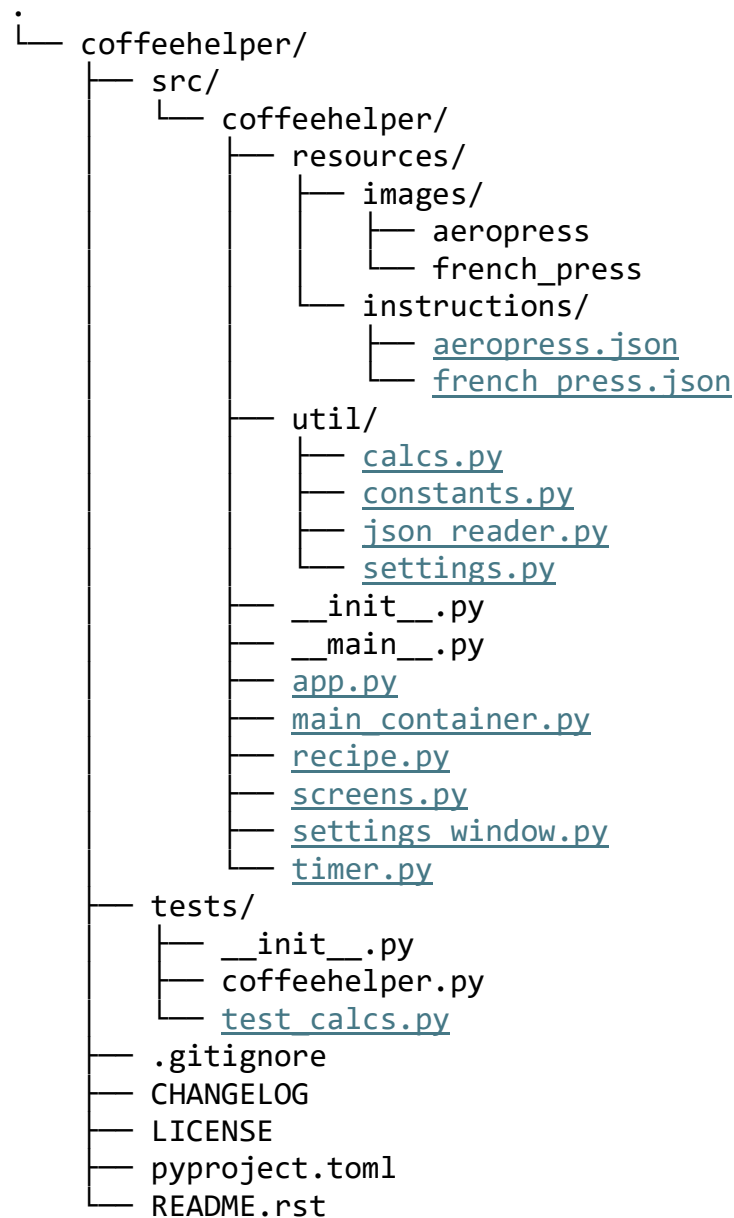
- Negative Zahlen: Wird vom Filter abgefangen.
- Sehr große Zahlen: Werden ohne Probleme gehandhabt.
- Sehr schnelles Klicken: Kein Problem.
- Kommas statt Punkte: Blockiert den Validator. (Intended behavior?)
- Leeres Feld führt zu „???“-Umwandlung, da es keine Zahl ist
- Schwierig, wieder zu Texteingabe zu kommen, wenn ??? eingetragen ist



13 Quellcodedokumentation

13.1 Dateistruktur

Im Folgenden ist zur Übersicht die Ordnerstruktur abgebildet. Bilddateien wurden in diese nicht aufgenommen. Alle nicht vollständig automatisch generierten Dateien folgen im Anschluss und sind hier verlinkt.





13.2 Code

13.2.1 app.py

```
1. """
2. A tool to help you prepare delicious coffee.
3. """
4.
5. import toga
6. from toga.style import Pack
7. from toga.style.pack import ROW
8.
9. from .main_container import MainContainer
10. from .settings_window import SettingsWindow
11. from .util.settings import settings
12.
13. class CoffeeHelper(toga.App):
14.     def startup(self):
15.         """Construct and show the Toga application."""
16.
17.         main_box = toga.Box(style=Pack(direction=ROW, alignment="center",
18. flex=1))
19.
20.         settings.load_config(self)
21.         main_container = MainContainer(app=self)
22.         main_box.add(main_container)
23.
24.         settings_window_cmd = toga.Command(
25.             self.open_settings_window, text="Settings",
26. group=toga.Group.FILE
27. )
28.
29.         self.commands.add(settings_window_cmd)
30.
31.         self.main_window = toga.MainWindow(
32.             title=self.formal_name, size=(300, 480), resizable=False
33. )
34.         self.main_window.content = main_box
35.         self.main_window.show()
36.
37.     def open_settings_window(self, command=None):
38.         """Creates and opens the settings window."""
39.         settings_window = SettingsWindow()
40.         settings_window.show()
41.
42. def main():
43.     return CoffeeHelper()
```



13.2.2 main_container.py

```

1. import toga
2. from toga.style import Pack
3. from toga.style.pack import COLUMN, ROW
4.
5. from .recipe import recipe
6. from .screens import (
7.     CoffeePreparationScreen,
8.     CoffeeSelectionScreen,
9.     FinishScreen,
10.    InstructionDisplayScreen,
11.    RatioSelectionScreen,
12. )
13. from .util.constants import (
14.     COFFEE_PREPARATION,
15.     COFFEE_SELECTION,
16.     FINISHED,
17.     INSTRUCTIONS,
18.     RATIO_SELECTION,
19. )
20. from .util.json_reader import JsonReader
21.
22. class MainContainer(toga.Box):
23.     """The main container which will display most of the user interface.
24.     Handles managing the various screens and switching between them."""
25.
26.     def __init__(self, app):
27.         super().__init__(style=Pack(direction=COLUMN, padding=10,
flex=1))
28.         self.app = app
29.
30.         self.current_step = 0
31.
32.         json_reader = JsonReader(self.app)
33.         self.instructions = json_reader.read_steps()
34.
35.         self.setup()
36.
37.     def setup(self):
38.         """Generates the overall layout and loads the initial screen."""
39.         self.main_box = toga.Box(style=Pack(flex=1))
40.         self.controls = toga.Box(style=Pack(direction=ROW, height=40,
flex=1))
41.
42.         self.add(self.main_box, self.controls)
43.         self.load_selection_screen()
44.
45.     # Element Loading Methods
46.
47.     def load_control_arrows(self):
48.         """Loads in the buttons that allow moving forward and backward
through the instructions."""
49.
50.         self.controls.clear()
51.         backward_button = toga.Button("<-",
on_press=self.backward_handler)

```



```

52.         spacer = toga.Box(
53.             style=Pack(flex=1)
54.         ) # ensures that the buttons "stick" to the left and right by
filling any remaining space.
55.         forward_button = toga.Button("Continue ->",
on_press=self.forward_handler)
56.         self.controls.add(backward_button, spacer, forward_button)
57.
58.     def load_selection_screen(self):
59.         """Loads the screen in which the user selects their coffee of
choice from the options available."""
60.
61.         self.current_step = COFFEE_SELECTION
62.         self.main_box.clear()
63.         self.coffee_selection_screen =
CoffeeSelectionScreen(self.instructions)
64.         self.main_box.add(self.coffee_selection_screen)
65.         self.controls.clear()
66.         start_button = toga.Button(
67.             "Start brewing!", style=Pack(flex=1),
on_press=self.start_button_handler
68.         )
69.         self.controls.add(start_button)
70.
71.     def load_ratio_screen(self):
72.         """Loads the screen in which the user chooses how much
water/coffee to brew."""
73.
74.         self.current_step = RATIO_SELECTION
75.         selected = self.coffee_selection_screen.selection.value
76.         instruction = None
77.
78.         # finds the instructions selected and loads them into memory
79.         for i in self.instructions:
80.             if i["name"] == selected.name:
81.                 instruction = i
82.                 break
83.
84.         recipe.load_recipe(instruction)
85.
86.         self.ratio_selection_screen = RatioSelectionScreen()
87.         self.main_box.clear()
88.         self.main_box.add(self.ratio_selection_screen)
89.
90.         self.load_control_arrows()
91.
92.     def load_coffee_preparation_screen(self):
93.         """Loads the screen which tells the user to weigh and grind their
coffee."""
94.
95.         # do not allow this screen to load unless water and coffee input
are valid
96.         if (
97.             not self.ratio_selection_screen.water_input.is_valid
98.             or not self.ratio_selection_screen.coffee_input.is_valid
99.         ):
100.             self.window.error_dialog(
101.                 "Error!", "Invalid input. Please only enter numbers."

```



```

102.         )
103.         return
104.
105.         # checking if input is valid (this should be caught by the
validator, this is a safety measure)
106.         try:
107.             recipe.water =
float(self.ratio_selection_screen.water_input.value)
108.             recipe.coffee =
float(self.ratio_selection_screen.coffee_input.value)
109.         except ValueError:
110.             self.window.error_dialog(
111.                 "Error!", "Error parsing input. Please only enter
numbers."
112.             )
113.             return
114.
115.             self.current_step = COFFEE_PREPARATION
116.             coffee_preparation_screen = CoffeePreparationScreen()
117.             self.main_box.clear()
118.             self.main_box.add(coffee_preparation_screen)
119.
120.         def load_instruction_display_screen(self):
121.             """Loads the screen that will guide the user through step-by-step
instructions."""
122.
123.             self.current_step = INSTRUCTIONS
124.             self.main_box.clear()
125.             self.instruction_display_screen =
InstructionDisplayScreen(self.app)
126.             self.main_box.add(self.instruction_display_screen)
127.
128.         def load_finish_screen(self):
129.             """Loads the screen that congratulates the user to their cup of
coffee."""
130.
131.             self.current_step = FINISHED
132.             self.main_box.clear()
133.             finish_screen = FinishScreen()
134.             self.main_box.add(finish_screen)
135.
136.             restart_button = toga.Button(
137.                 "Brew more coffee!",
138.                 style=Pack(flex=1),
139.                 on_press=self.restart_button_handler,
140.             )
141.             self.controls.clear()
142.             self.controls.add(restart_button)
143.
144.         # Event Handlers
145.
146.         def forward_handler(self, widget=None):
147.             """Handles presses of the forward button.
148.             Determines the current step and performs the appropriate
action."""
149.
150.             if self.current_step == RATIO_SELECTION:
151.                 self.load_coffee_preparation_screen()

```



```
152.         elif self.current_step == COFFEE_PREPARATION:
153.             self.load_instruction_display_screen()
154.         elif self.current_step == INSTRUCTIONS:
155.             result = self.instruction_display_screen.next_step()
156.             if result is False:
157.                 self.current_step == FINISHED
158.                 self.load_finish_screen()
159.         else:
160.             print("Forward pressed.")
161.
162.     def backward_handler(self, widget=None):
163.         """Handles presses of the backward button.
164.         Determines the current step and performs the appropriate
165.         action."""
166.         if self.current_step == RATIO_SELECTION:
167.             self.load_selection_screen()
168.         elif self.current_step == COFFEE_PREPARATION:
169.             self.load_ratio_screen()
170.         elif self.current_step == INSTRUCTIONS and recipe.current_step <=
171.             0:
172.             self.load_coffee_preparation_screen()
173.         elif self.current_step == INSTRUCTIONS:
174.             self.instruction_display_screen.previous_step()
175.         else:
176.             print("Backward pressed.")
177.
178.     def start_button_handler(self, widget=None):
179.         """Handles presses of the start button."""
180.         self.load_ratio_screen()
181.
182.     def restart_button_handler(self, widget=None):
183.         """Handles presses of the restart button."""
184.         self.load_selection_screen()
```



13.2.3 recipe.py

```
1. from .util import constants
2.
3. class Recipe:
4.     """Class representing the currently selected recipe.
5.     Do not manually instantiate this, simply import the instance from this
6.     module."""
7.
8.     def load_recipe(self, instructions):
9.         """Receives a dictionary representation of a given recipe
10.         and uses it to initialize/update all attributes."""
11.
12.         self.name = instructions.get("name")
13.         self.image_folder = instructions.get("image_folder", "/")
14.         self.grind = instructions.get("grind", constants.DEFAULT_GRIND)
15.         ratio = instructions.get("ratio")
16.         if ratio:
17.             self.water = ratio.get("water", constants.DEFAULT_WATER)
18.             self.coffee = ratio.get("coffee", constants.DEFAULT_COFFEE)
19.         else:
20.             self.water = constants.DEFAULT_WATER
21.             self.coffee = constants.DEFAULT_COFFEE
22.         self.steps = instructions.get("steps")
23.         self.current_step = 0
24.
25.     def next_step(self):
26.         self.current_step += 1
27.
28.     def previous_step(self):
29.         self.current_step -= 1
30.
31.     def get_current_step(self):
32.         """Returns the contents of the current step.
33.         Returns None when the final step is reached."""
34.
35.         if self.current_step < len(self.steps):
36.             return self.steps[self.current_step]
37.         else:
38.             return None
39.
40. # singleton instance
41. recipe = Recipe()
```



13.2.4 screens.py

```

1. from pathlib import Path
2.
3. import toga
4. from toga.style import Pack
5. from toga.style.pack import COLUMN, ROW
6. from toga.validators import Number
7.
8. from .recipe import recipe
9. from .timer import Timer
10. from .util import calcs
11. from .util.settings import settings
12.
13. class ImageTextBox(toga.Box):
14.     """Base class for creating a box with an image at the top and some
text at the bottom."""
15.
16.     def __init__(self):
17.         super().__init__(style=Pack(flex=1, direction=COLUMN, padding=(5,
0)))
18.         self.image_box = toga.Box(style=Pack(direction=COLUMN, flex=1,
padding=5))
19.         self.label_box = toga.Box(style=Pack(direction=COLUMN,
padding=(5, 0)))
20.         self.add(self.image_box, self.label_box)
21.
22. class CoffeeSelectionScreen(toga.Box):
23.     """Class representing the screen in which the users selects the
coffee they want to prepare.
24.
25.     Takes a dictionary representation of instructions as an argument."""
26.
27.     def __init__(self, instructions):
28.         super().__init__(style=Pack(direction=COLUMN, alignment="center",
flex=1))
29.         self.instructions = instructions
30.         self.setup()
31.
32.     def setup(self):
33.         label = toga.Label(
34.             "Which method would you like to use?",
style=Pack(padding=(10, 0, 10, 0))
35.         )
36.         self.selection = toga.Selection(items=self.instructions,
accessor="name")
37.         self.add(label, self.selection)
38.
39. class RatioSelectionScreen(toga.Box):
40.     """Class representing the screen in which the user chooses how much
coffee to prepare."""
41.
42.     def __init__(self):
43.         super().__init__(style=Pack(direction=COLUMN, flex=1))
44.         self.changing = (
45.             False # helper variable to avoid infinitely looping field
updates

```



```

46.         )
47.         self.ratio = calcs.calculate_ratio(recipe.coffee, recipe.water)
48.
49.         self.setup()
50.
51.     def setup(self):
52.         """Generates the user interface."""
53.
54.         row1 = toga.Box(style=Pack(direction=ROW, padding=5))
55.         water_label = toga.Label("Water", style=Pack(width=100))
56.         self.water_input = toga.TextInput(
57.             value=recipe.water,
58.             on_change=self.on_water_change_handler,
59.             validators=[Number(allow_empty=False)],
60.         )
61.         ml_label = toga.Label("ml")
62.         row1.add(water_label, self.water_input, ml_label)
63.
64.         row2 = toga.Box(style=Pack(direction=ROW, padding=5))
65.         coffee_label = toga.Label("Coffee", style=Pack(width=100))
66.         self.coffee_input = toga.TextInput(
67.             value=recipe.coffee,
68.             on_change=self.on_coffee_change_handler,
69.             validators=[Number()],
70.         )
71.         gram_label = toga.Label("g")
72.         row2.add(coffee_label, self.coffee_input, gram_label)
73.
74.         row3 = toga.Box(style=Pack(padding=5, direction=ROW))
75.         reset_button = toga.Button(
76.             "Reset",
77.             style=Pack(flex=1),
78.             on_press=self.reset_button_handler,
79.         )
80.         row3.add(reset_button)
81.
82.         self.add(row1, row2, row3)
83.
84.     # Event handlers
85.
86.     def on_coffee_change_handler(self, widget: toga.TextInput):
87.         if not self.changing:
88.             try:
89.                 self.changing = True
90.                 coffee = float(
91.                     widget.value or 0 # Prevent empty string from
causing an Error
92.                 )
93.                 water = round(calcs.calculate_water_from_coffee(coffee,
self.ratio))
94.                 self.water_input.value = water
95.             except ValueError:
96.                 widget.value = ""
97.                 widget.placeholder = "???"
98.                 self.changing = False
99.
100.    def on_water_change_handler(self, widget: toga.TextInput):
101.        if not self.changing:

```




```

102.         try:
103.             self.changing = True
104.             water = float(
105.                 widget.value or 0 # Prevent empty string from
causing an Error
106.             )
107.             coffee = round(calcs.calculate_coffee_from_water(water,
self.ratio), 1)
108.             self.coffee_input.value = coffee
109.         except ValueError:
110.             widget.value = ""
111.             widget.placeholder = "???"
112.             self.changing = False
113.
114.     def reset_button_handler(self, widget):
115.         self.coffee_input.value = recipe.coffee
116.         self.water_input.value = recipe.water
117.
118. class CoffeePreparationScreen(ImageTextBox):
119.     """Class representing the screen in which the user is told to weigh
and grind their coffee."""
120.
121.     def __init__(self):
122.         super().__init__()
123.         self.has_grinder = settings.get_coffee_grinder()
124.         self.load_content()
125.
126.     def load_content(self):
127.         """Loads the UI elements."""
128.
129.         if self.has_grinder is None:
130.             self.setup_grinder()
131.         elif self.has_grinder:
132.             try:
133.                 image = toga.Image("resources/images/grinder.png")
134.                 image_view = toga.ImageView(image, style=Pack(flex=1))
135.                 self.image_box.add(image_view)
136.             except OSError as e:
137.                 print("Image loading failed. Error:", e)
138.
139.             label = toga.Label(
140.                 f"Weigh and grind {recipe.coffee} grams of coffee.\n"
141.                 f"Grind setting: {recipe.grind}"
142.             )
143.             self.label_box.add(label)
144.
145.         else:
146.             try:
147.                 image = toga.Image("resources/images/scale.png")
148.                 image_view = toga.ImageView(image, style=Pack(flex=1))
149.                 self.image_box.add(image_view)
150.             except OSError as e:
151.                 print("Image loading failed. Error:", e)
152.
153.             label = toga.Label(
154.                 f"Weigh {recipe.coffee} grams of preground coffee.\n\n"
155.                 f"Preferred grind level: {recipe.grind}"
156.             )

```



```

157.         self.label_box.add(label)
158.
159.     def setup_grinder(self):
160.         """Loads the screen where you select whether you have a grinder
or not."""
161.
162.         try:
163.             image = toga.Image("resources/images/grinder.png")
164.             image_view = toga.ImageView(image, style=Pack(flex=1))
165.         except OSError as e:
166.             image_view = None
167.             print("Image loading failed. Error:", e)
168.
169.         label = toga.Label("Do you have a coffee grinder?")
170.         answers = toga.Box(style=Pack(direction=ROW, padding=5))
171.         yes_button = toga.Button(
172.             "Yes", style=Pack(flex=1), on_press=self.yes_button_handler
173.         )
174.         no_button = toga.Button(
175.             "No", style=Pack(flex=1), on_press=self.no_button_handler
176.         )
177.         self.image_box.add(image_view)
178.         answers.add(yes_button, no_button)
179.         self.image_box.add(image_view)
180.         self.label_box.add(label, answers)
181.
182.     def yes_button_handler(self, widget=None):
183.         settings.set_coffee_grinder(True)
184.         self.has_grinder = True
185.         self.image_box.clear()
186.         self.label_box.clear()
187.         self.load_content()
188.
189.     def no_button_handler(self, widget=None):
190.         settings.set_coffee_grinder(False)
191.         self.has_grinder = False
192.         self.image_box.clear()
193.         self.label_box.clear()
194.         self.load_content()
195.
196. class InstructionDisplayScreen(ImageTextBox):
197.     """Represents the screen in which the user is lead through the step-
by-step instructions to make coffee."""
198.
199.     def __init__(self, app): # app is needed for access to the event
loop
200.         super().__init__()
201.         self.app = app
202.         self.timer = None
203.         self.step: dict = recipe.get_current_step()
204.         self.setup()
205.
206.     def setup(self):
207.         """Creates the containers for the user interface."""
208.
209.         self.timer_box = toga.Box(style=Pack(padding=(5, 0)))
210.         self.add(self.timer_box)
211.         self.load_step()

```



```

212.
213.     def load_step(self):
214.         """Loads the current step into the user interface,
215.         adding an image and a timer as needed."""
216.
217.         # clear any previous steps
218.         self.image_box.clear()
219.         self.label_box.clear()
220.         self.timer_box.clear()
221.
222.         # stop previous timer, if any
223.         if self.timer:
224.             self.timer.stop()
225.
226.         image = self.step.get("image")
227.         text = self.step.get("text")
228.         timer = self.step.get("timer")
229.
230.         if image:
231.             self.load_image(image)
232.         if text:
233.             self.load_text(text)
234.         if timer:
235.             self.load_timer(timer)
236.
237.     def load_text(self, text):
238.         label = toga.Label(text.format(coffee=recipe.coffee,
239. water=recipe.water))
240.         self.label_box.add(label)
241.
242.     def load_image(self, image):
243.         try:
244.             image_obj = toga.Image(
245.                 Path("resources/images") / recipe.image_folder / image
246.             )
247.             image_view = toga.ImageView(image_obj, style=Pack(flex=1))
248.             self.image_box.add(image_view)
249.         except (FileNotFoundError, ValueError):
250.             print("Error: Image", image, "not found.")
251.
252.     def load_timer(self, time):
253.         self.timer = Timer(self.app, time)
254.         self.timer_box.add(self.timer)
255.
256.     def next_step(self):
257.         """Gets the next step from the recipe and initializes loading it.
258.         Returns True if there is another step to get, otherwise False."""
259.         recipe.next_step()
260.         self.step = recipe.get_current_step()
261.         if self.step:
262.             self.load_step()
263.             return True
264.         else:
265.             if self.timer:
266.                 self.timer.stop()
267.             return False
268.
269.     def previous_step(self):

```



```
269.         "Gets the previous step from the recipe and initializes loading
it."
270.         recipe.previous_step()
271.         self.step = recipe.get_current_step()
272.         self.load_step()
273.
274. class FinishScreen(ImageTextBox):
275.     """Represents the screen after the user finishes preparing their
coffee."""
276.
277.     def __init__(self):
278.         super().__init__()
279.         self.setup()
280.
281.     def setup(self):
282.         """Loads the UI elements."""
283.         try:
284.             image = toga.Image("resources/images/coffee_cup.png")
285.             image_view = toga.ImageView(image, style=Pack(flex=1))
286.             self.image_box.add(image_view)
287.         except OSError as e:
288.             print("Unable to load image. Error:", e)
289.             label = toga.Label("Enjoy your coffee!")
290.             self.label_box.add(label)
291.
```



13.2.5 settings_window.py

```
1. import toga
2. from toga.style import Pack
3. from toga.style.pack import COLUMN
4.
5. from .util.settings import settings
6.
7. class SettingsWindow(toga.Window):
8.     """A window to edit the settings."""
9.
10.    def __init__(self):
11.        super().__init__(size=(210, 100), title="Settings")
12.        self.content = toga.Box(style=Pack(direction=COLUMN, padding=20))
13.        self.setup()
14.
15.    def setup(self):
16.        """Creates the user interface."""
17.        has_grinder = settings.get_coffee_grinder()
18.        self.grinder_checkbox = toga.Switch(
19.            # if None (unset), convert to False for unchecked box
20.            "Has coffee grinder",
21.            value=bool(has_grinder),
22.            style=Pack(padding=(5, 0)),
23.        )
24.        save_button = toga.Button("Save settings",
on_press=self.save_settings)
25.        self.content.add(self.grinder_checkbox, save_button)
26.
27.    def save_settings(self, widget=None):
28.        """Reads out the settings and saves them into the config file."""
29.        has_grinder = self.grinder_checkbox.value
30.        settings.set_coffee_grinder(has_grinder)
31.        self.close()
32.
```



13.2.6 timer.py

```

1. import asyncio
2. import time
3.
4. import toga
5. from toga.style import Pack
6. from toga.style.pack import COLUMN, ROW
7.
8. class Timer(toga.Box):
9.     """Class that represents a timer module with a progress bar and a
countdown."""
10.
11.     def __init__(self, app: toga.App, duration_in_seconds: int):
12.         """Constructs a new timer.
13.
14.         Parameters
15.         -----
16.         app: toga.App
17.             the app the Timer belongs to. This is required for access to
the event loop.
18.         duration_in_seconds: int
19.             the duration the timer is supposed to run.
20.         """
21.         super().__init__(style=Pack(direction=COLUMN, flex=1))
22.         self.app = app
23.         self.duration = duration_in_seconds
24.         self.pause_progress = 0
25.         self.setup()
26.
27.     def setup(self):
28.         """Generates the UI."""
29.         timer_row = toga.Box(style=Pack(direction=ROW, flex=1))
30.         self.progress_bar = toga.ProgressBar(
31.             max=self.duration, value=0, style=Pack(flex=1)
32.         )
33.
34.         self.start_button = toga.Button("Start", on_press=self.start)
35.         self.pause_button = toga.Button("Pause", enabled=False)
36.
37.         timer_row.add(self.progress_bar, self.start_button,
self.pause_button)
38.         self.add(timer_row)
39.
40.         self.label = toga.Label(
41.             f"Time: {self.duration // 60:02d}:{self.duration % 60:02d}"
42.         )
43.         self.add(self.label)
44.
45.     def update_ui(self, time):
46.         """Updates the progress bar and time display with the remaining
time once each tick."""
47.
48.         self.progress_bar.value = time
49.         time_remaining = int(
50.             self.duration - time

```



```

51.         ) # truncating instead of rounding, because the second should
only tick up once it's full
52.         self.label.text = f"Time: {time_remaining //
60:02d}:{time_remaining % 60:02d}"
53.
54.     def start(self, widget=None):
55.         """Starts the timer."""
56.
57.         self.start_button.text = "Cancel"
58.         self.start_button.on_press = self.abort
59.
60.         self.pause_button.text = "Pause"
61.         self.pause_button.on_press = self.pause
62.         self.pause_button.enabled = True
63.         self.pause_button.focus()
64.
65.         self.progress_bar.start()
66.         self.start_time = time.time()
67.         self.is_running = True
68.         self.app.add_background_task(self.tick_timer)
69.
70.     def stop(self, widget=None):
71.         """Stops the timer."""
72.
73.         self.start_button.text = "Start"
74.         self.start_button.on_press = self.start
75.         self.pause_button.enabled = False
76.
77.         self.is_running = False
78.         self.progress_bar.stop()
79.
80.     def pause(self, widget=None):
81.         """Pauses the timer and saves the current progress."""
82.
83.         self.pause_button.text = "Resume"
84.         self.pause_button.on_press = self.resume
85.
86.         self.is_running = False
87.         self.pause_progress = time.time() - self.start_time
88.         self.progress_bar.stop()
89.
90.     def resume(self, widget=None):
91.         """Resumes the timer where it was left off."""
92.
93.         self.pause_button.text = "Pause"
94.         self.pause_button.on_press = self.pause
95.
96.         self.is_running = True
97.         self.start_time = time.time() - self.pause_progress
98.         self.app.add_background_task(self.tick_timer)
99.
100.    def abort(self, widget=None):
101.        """Cancels the timer and resets it."""
102.
103.        self.update_ui(0)
104.        self.stop()
105.
106.    def finish(self, widget=None):

```



```
107.         """Stops the timer and ensures the progress bar is fully
filled."""
108.
109.         self.progress_bar.value = self.duration
110.         self.app.beep()
111.         self.stop()
112.
113.     async def tick_timer(self, app):
114.         """Coroutine that gets added to the app's main event loop.
115.         Twice a second, it checks how much time has passed since the
timer has started.
116.         If enough time has passed for the timer to finish, the finish
method is called.
117.         Otherwise, updates the UI with the current remaining time and
asks the event loop to return in half a second."""
118.
119.         while self.is_running:
120.             delta = time.time() - self.start_time
121.             if delta >= self.duration:
122.                 self.finish()
123.                 return
124.
125.             self.update_ui(delta)
126.             # asynchronous sleep to prevent blocking the UI
127.             await asyncio.sleep(0.5)
128.
```

13.2.7 calcs.py

```
1.  """Helper functions for various calculations."""
2.
3.  def calculate_ratio(coffee: int, water: int) -> float:
4.      try:
5.          return coffee / water
6.      except ZeroDivisionError:
7.          return 0
8.
9.  def calculate_coffee_from_water(water: int, ratio: float) -> float:
10.     return water * ratio
11.
12.  def calculate_water_from_coffee(coffee: int, ratio: float) -> float:
13.      try:
14.          return coffee / ratio
15.      except ZeroDivisionError:
16.          return 0
17.
```




13.2.8 constants.py

```

1. """Storing various constant values for consistency and easy
modification."""
2.
3. # default values
4.
5. DEFAULT_WATER = 500
6. DEFAULT_COFFEE = 30
7. DEFAULT_GRIND = "medium"
8.
9. # Preparation steps
10.
11. COFFEE_SELECTION = 0
12. RATIO_SELECTION = 1
13. COFFEE_PREPARATION = 2
14. INSTRUCTIONS = 3
15. FINISHED = 4
16.

```

13.2.9 json_reader.py

```

1. import json
2. import os
3.
4. import toga
5.
6. class JsonReader:
7.     """Helper class to import the instruction JSON files."""
8.
9.     def __init__(self, app: toga.App):
10.         self.app = app
11.
12.     def read_steps(self) -> list[dict]:
13.         """Reads out the instruction files and returns them as a list of
dictionaries."""
14.
15.         output = []
16.
17.         # read all instruction files
18.         try:
19.             for file in os.scandir(self.app.paths.app /
"resources/instructions"):
20.                 if file.is_file and file.path.endswith(".json"):
21.                     with open(file.path) as f:
22.                         output.append(json.load(f))
23.         except OSError as e:
24.             print("Error getting files.", e, type(e))
25.             return None
26.
27.         # verify if instruction files are valid and only return the valid
ones
28.         return [
29.             instructions
30.             for instructions in output

```



```

31.         if self.verify_valid_instructions(instructions)
32.     ]
33.
34.     def verify_valid_instructions(self, instruction: dict) -> bool:
35.         """Does some soft verification of the validity of the instruction
36.         files.
37.         This is not bulletproof; always ensure the instruction files are
38.         valid."""
39.         if "name" not in instruction or type(instruction["name"]) is not
40.         str:
41.             return False
42.         if "steps" not in instruction or type(instruction["steps"]) is not
43.         list:
44.             return False
45.         for step in instruction["steps"]:
46.             if "text" not in step:
47.                 return False
48.             if "timer" in step and type(step["timer"]) is not int:
49.                 return False
50.         return True
51.

```

13.2.10 settings.py

```

1. import configparser
2. import toga
3.
4. class Settings:
5.     """Class representing the settings of the application.
6.     Do not manually instantiate this, simply import the instance from this
7.     module."""
8.     def __init__(self):
9.         """Initializes the config file."""
10.        # load settings file or load defaults and maybe create settings
11.        file here
12.        self.app = None
13.        self.config = configparser.ConfigParser()
14.
15.    def load_config(self, app: toga.App):
16.        """Loads the current settings file.
17.        Requires the app parameter to know where to look for it."""
18.        self.app = app
19.
20.        try:
21.            self.config.read(self.app.paths.config / "config.ini")
22.        except FileNotFoundError:
23.            print("File not found.")
24.
25.    def write_config(self) -> bool:
26.        """Writes the current state of the config into the settings file.

```



```

27.         Returns True if the operation is successful, otherwise False"""
28.
29.     try:
30.         self.app.paths.config.mkdir(
31.             parents=True, exist_ok=True
32.         ) # create path if it doesn't exist
33.         with open(self.app.paths.config / "config.ini", "w") as f:
34.             self.config.write(f)
35.         return True
36.     except OSError as e:
37.         print("Error writing file. Error:", e)
38.         return False
39.
40.     def get_coffee_grinder(self) -> bool | None:
41.         """Parses the config file to determine if the user owns a coffee
42. grinder.
43. Returns the value of the key if it exists, or None if it is
44. unset."""
45.         if self.config.has_section("Hardware"):
46.             return self.config["Hardware"].getboolean("CoffeeGrinder",
47. None)
48.         else:
49.             return None
50.
51.     def set_coffee_grinder(self, value: bool):
52.         """Updates the config file and sets wether the user has a coffee
53. grinder or not."""
54.         if not self.config.has_section("Hardware"):
55.             self.config["Hardware"] = {}
56.             self.config["Hardware"]["CoffeeGrinder"] = str(value)
57.             self.write_config()
58.
59. # Singleton Instance
60. settings = Settings()
61.

```

13.2.11 test_calcs.py

```

1. """Unit Tests"""
2.
3. from coffeehelper.util import calcs
4.
5. # Ratio
6.
7. def test_ratio_1_1():
8.     assert calcs.calculate_ratio(1, 1) == 1.0
9.
10. def test_ratio_50_10():
11.     assert calcs.calculate_ratio(50, 10) == 5.0
12.
13. def test_ratio_10_50():
14.     assert calcs.calculate_ratio(10, 50) == 0.2
15.
16. def test_ratio_0_0():

```



```
17.     """Testing against zero devision errors."""
18.     assert calcs.calculate_ratio(0, 0) == 0
19.
20. # Calculating coffee from water
21.
22. def test_coffee_from_water_1_1():
23.     assert calcs.calculate_coffee_from_water(1, 1) == 1
24.
25. def test_coffee_from_water_500_0_5():
26.     assert calcs.calculate_coffee_from_water(500, 0.5) == 250
27.
28. def test_coffee_from_water_0_0():
29.     assert calcs.calculate_coffee_from_water(0, 0) == 0
30.
31. # Calculating water from coffee
32.
33. def test_water_from_coffee_1_1():
34.     assert calcs.calculate_water_from_coffee(1, 1) == 1
35.
36. def test_water_from_coffee_500_0_5():
37.     assert calcs.calculate_water_from_coffee(500, 0.5) == 1000
38.
39. def test_water_from_coffee_0_0():
40.     """Testing against zero division errors."""
41.     assert calcs.calculate_water_from_coffee(0, 0) == 0
42.
```



13.2.12 aeropress.json

```
1. {
2.   "name": "Aeropress",
3.   "image_folder": "aeropress",
4.   "grind": "medium to fine",
5.   "ratio": {
6.     "water": 200,
7.     "coffee": 11
8.   },
9.   "steps": [
10.    {
11.      "text": "Put the paper filter into your Aeropress and close
12.      it.\nPlace it on a mug or a similar container.",
13.      "image": "ae_filter.jpg"
14.    },
15.    {
16.      "text": "Add {coffee} grams of coffee to your aeropress",
17.      "image": "ae_add_coffee.jpg"
18.    },
19.    {
20.      "text": "Add {water} ml of water to your aeropress.\nUsing a
21.      scale is recommended.",
22.      "image": "ae_add_water.jpg"
23.    },
24.    {
25.      "text": "Insert plunger slightly and wait for 2.5 minutes.",
26.      "image": "ae_steep.jpg",
27.      "timer": 150
28.    },
29.    {
30.      "text": "Gently push the plunger all the way to the bottom.",
31.      "image": "ae_plunge.jpg"
32.    }
33.  ]
34. }
```



13.2.13 frenchpress.json

```
1. {  
2.   "name": "French Press",  
3.   "image_folder": "french_press",  
4.   "grind": "medium to coarse",  
5.   "ratio": {  
6.     "water": 500,  
7.     "coffee": 30  
8.   },  
9.   "steps": [  
10.    {  
11.      "text": "Add {coffee} grams of coffee to your French Press.",  
12.      "image": "fp_add_coffee.jpg"  
13.    },  
14.    {  
15.      "text": "Add {water} grams of boiling water.",  
16.      "image": "fp_add_water.jpg"  
17.    },  
18.    {  
19.      "text": "Let the coffee steep for 4 minutes.",  
20.      "image": "fp_steep.jpg",  
21.      "timer": 240  
22.    },  
23.    {  
24.      "text": "Add the lid and very slowly push the plunger down.",  
25.      "image": "fp_plunge.jpg"  
26.    },  
27.    {  
28.      "text": "Very slowly pour the coffee into cups.",  
29.      "image": "fp_pour.jpg"  
30.    }  
31.  ]  
32. }  
33.
```

Abnahmeprotokoll

Nr.	Feature	Umgesetzt
1.	Benutzerfreundliche GUI	✓
2.	Zubereitung French Press, Aeropress	✓
3.	Berechnung Kaffeemenge aus Wassermenge	✓
4.	Berechnung Wassermenge aus Kaffeemenge	✓
5.	Reset-Button für die Mengenauswahl	✓
6.	Step-By-Step-Anweisungen für die Zubereitung	✓
7.	Timer für Wartezeiten	✓
8.	Konfiguration Kaffeemühle (vorhanden/nicht vorhanden)	✓
9.	Abfangen von nicht-numerischen Eingaben (Mengeneingabe)	✓
10.	Timer pausieren und neu starten	✓
11.	Neustarten der Zubereitung nach Ende	✓
12.	Installation mit Installationswizard	✓

A 9 Meldung über eine Fehlermeldung